ÖNSÖZ

Değerli öğrencilerimiz,

Bilişim teknolojilerinin baş döndürücü bir hızla geliştiği ve hayatımızın her alanına nüfuz ettiği günümüzde, yazılım dillerini anlamak ve bilgisayarlarla etkin bir iletişim kurabilmek en değerli yetkinliklerden biri haline gelmiştir. Hayatımızı kolaylaştıran otomasyonlardan, cebimizdeki akıllı cihazlara kadar her şeyin arkasında sağlam bir algoritma mantığı yatmaktadır. "Algoritma ve Programlamaya Giriş" dersi, tam da bu uçsuz bucaksız dijital dünyanın kapılarını aralayan temel bir anahtar niteliğindedir.

Elinizdeki bu eğitim dokümanı, meslek yüksekokulu önlisans seviyesindeki öğrencilerin gereksinimlerine ve müfredatına özel olarak, adım adım ve anlaşılır bir hiyerarşiyle kurgulanmıştır. Asıl amacımız, sadece sizlere kod yazmayı öğretmek değil; karşılaştığınız zorlukları analitik bir süzgeçten geçirerek mantıksal çözüm yolları (algoritmalar) üretebilme kültürünü zihninize işlemektir. Donanım ve yazılımın temel felsefesinden başlayarak, akış diyagramlarıyla görsel modellemelere, oradan da yapısal programlamanın atası kabul edilen C programlama dilinin derinliklerine kadar uzanan 14 haftalık verimli bir akademik yolculuğa çıkacağız.

Unutmayın ki programlama öğrenmek, yepyeni bir yabancı dil öğrenmek gibidir; bol bol pratik yapmayı, denemekten çekinmemeyi, sabretmeyi ve en önemlisi hata yapmaktan korkmamayı gerektirir. Karşılaştığınız her bir "hata" mesajı, aslında doğrunun nasıl yapılması gerektiğini gösteren değerli birer tecrübedir. Bu süreçte pes etmeden, hevesle ve azimle araştırmaya devam etmelisiniz.

Hazırladığımız bu kaynakların kariyer yolculuğunuzda sizlere sağlam bir temel inşa etmesini umut ediyor; analitik düşünceyle aydınlanacağınız başarılı, heyecan verici ve keyifli bir eğitim dönemi diliyorum.

Öğr. Gör. İbrahim AYAZ

Hafta 1: Genel Programlama Bilgisi, Programlamanın Temelleri ve Bilgisayarın Temelleri

1. Giriş ve Ön Hazırlık

Bu haftaki dersimiz, yazılım geliştirme dünyasına ilk adımınızı atmanızı sağlayacaktır. Programlamayı öğrenmeden önce, kodlarımızı çalıştıracak olan temel aracımız "bilgisayarı" ve "çalışma mantığını" anlamamız son derece önemlidir. Önlisans seviyesine uygun olarak hazırlanmış bu dökümanda adım adım ve hiyerarşik bir dil kullanılmaktadır.

Hedefler: - Bilgisayarın temel bileşenlerini (donanım ve yazılım) tanımak. - Bilgisayarların nasıl çalıştığı ve veriyi nasıl işlediği hakkında temel bilgiye sahip olmak. - Programlama, yazılım, kodlama kelimelerinin teknik karşılıklarını öğrenmek. - Bilgisayar dillerinin seviyeleri hakkında fikir sahibi olmak.

---

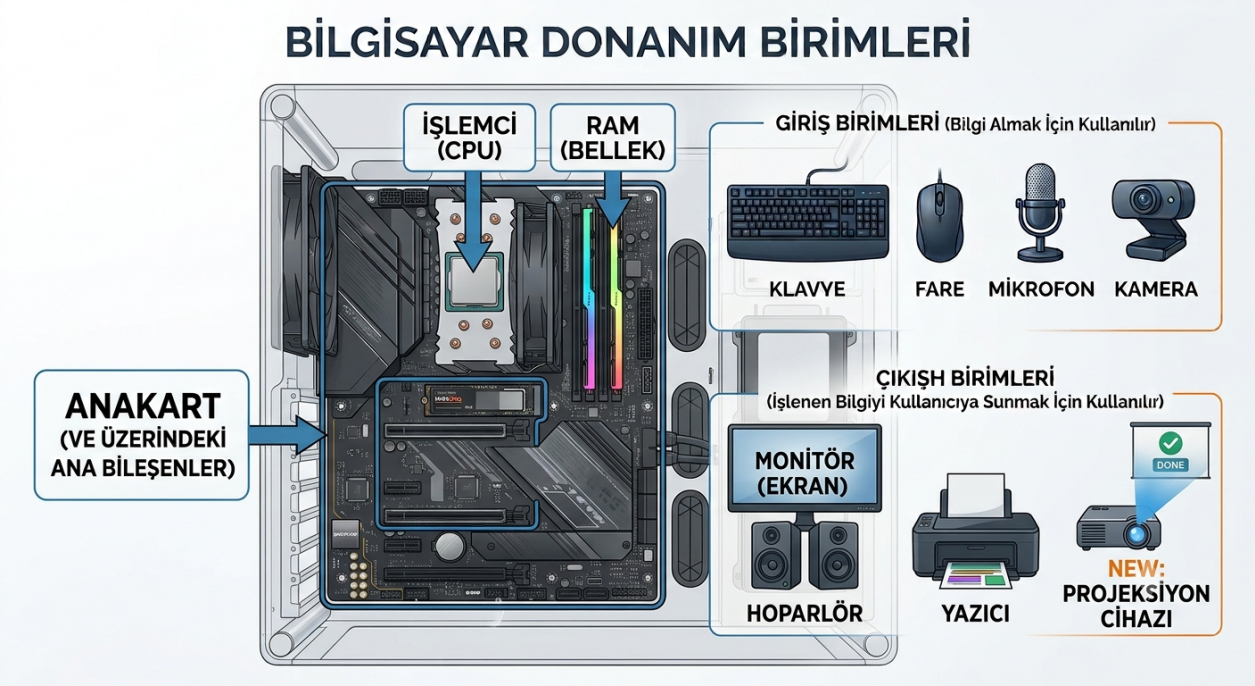
2. Bilgisayarın Temelleri

Bilgisayar; kendisine verilen verileri (girdileri) alan, kendi içindeki kurallara ve programlara göre işleyip anlamlı sonuçlar (çıktılar) üreten ve bu verileri depolayabilen elektronik bir cihazdır. Bir bilgisayarın çalışma prensibini insan beynine benzetebiliriz. Çevreden gelen uyarıları alırız, beyinde işleriz ve bir eylemle tepki veririz.

Bilgisayar temelde iki ana unsurdan oluşur: **Donanım (Hardware)** ve **Yazılım (Software)**.

2.1. Donanım (Hardware)

Donanım, bilgisayarın elle tutulup gözle görülebilen fiziksel parçalarıdır. - **İşlemci (CPU):** Bilgisayarın beynidir. Tüm matematiksel ve mantıksal işlemler burada yapılır. - **Bellek (RAM - Random Access Memory):** İşlemcinin anlık olarak ihtiyaç duyduğu verilerin geçici olarak tutulduğu yerdir. Elektrik kesildiğinde buradaki veriler silinir. - **Sabit Disk / SSD:** Bilgilerimizin kalıcı olarak depolandığı yerdir. Cihaz kapansa dahi verileriniz uçar gitmez. - **Giriş Birimleri:** Fare, klavye, mikrofon, kamera. (Bilgi almak için kullanılır.) - **Çıkış Birimleri:** Monitör (Ekran), hoparlör, yazıcı. (İşlenen bilgiyi kullanıcıya sunmak için kullanılır.)



2.2. Yazılım (Software)

Yazılım, bilgisayarın donanımlarını kontrol eden ve belirli bir amaca yönelik çalışan yönnergeler bütünü veya kodlar kümesidir. Donanım sadece cihazın ruhsuz bir iskeletiyken, yazılım ona can veren zekadır. - **Sistem Yazılımları:** Windows, macOS, Linux, Android gibi İşletim Sistemleri. - **Uygulama Yazılımları:** Word, Excel, Chrome Tarayıcı, Video oyunları.



3. Programlamanın Temelleri

Gelelim ana konumuz olan programlamaya.

3.1. Programlama Nedir?

Programlama (veya kodlama), bir problemi çözmek için bilgisayara ne yapması gerektiğini anlatan komutlar dizisi yazma işidir. İnsanın başka bir insanla anlaşması için doğal dilleri (Türkçe, İngilizce vb.) kullanması gibi, insanların da bilgisayarlarla iletişim kurması için **Programlama Dilleri** geliştirmesi gerekmiştir.

Neden Programlamaya İhtiyacımız Var? - Tekrar eden rutin işleri saniyeler içinde otomatik olarak yaptırmak. - İnsan gücü ile yıllar sürecek matematiksel veya istatistiksel hesaplamaları saniyeler içinde yapmak. - Hayatımızı kolaylaştıran modern uygulamalar (e-devlet, bankacılık uygulamaları, hastane otomasyonları) oluşturmak.

3.2. Bilgisayar Veriyi Nasıl Anlar? (0 ve 1 Kavramı)

Bilgisayarların içinde sadece elektronik devreler ve elektrik sinyalleri vardır. Bu nedenle bir bilgisayar, Türkçe veya İngilizce bilmez. Yalnızca makine dili dediğimiz **0 ve 1 (Binary - İkili sistem)** mantığıyla çalışır. - **1:** Elektrik var (True / Doğru) - **0:** Elektrik yok (False / Yanlış)

Siz klavyeden "A" harfine bastığınızda arka planda işlemciye (örneğin 01000001) şeklinde bir ikili kod gider, bu işlemden geçer ve ekranda A olarak görünüz.

3.3. Programlama Dillerinin Seviyeleri

Her defasında 0 ve 1'leri kullanarak yazılım üretmek bir insan için işkencedir. Bu engeli aşmak için çeşitli seviyelerde diller oluşturulmuştur:

1. **Makine Dili (Düşük Seviye):** Sadece 0 ve 1'ler. Bilgisayar doğrudan anlar ancak insanın yazması neredeyse imkansızdır.
2. **Assembly Dili:** Makine dilinin biraz daha insancıllaştırılmış halidir, `ADD`, `MOV` gibi kısa kısaltmalar içerir. Ancak yine de yazması zordur.
3. **Orta Seviye Diller:** Hem makineden anlayan hem de insan diline yakın olan dillerdir. Özellikle donanım kontrolü yaparken işimize yarar. (Örn: C dili)
4. **Yüksek Seviye Diller:** İnsan diline, öğrenmesine oldukça yakın olan dillerdir. Okunabilirliği yüksektir. (Örn: Python, Java, C#)



Bizim bu dönemin ilerleyen haftalarında öğreneceğimiz **C Dili**, daha çok orta-yüksek seviye arasında konumlandırılır ve temel kodlama mantığını öğrenmek için dünya üzerinde en çok tercih edilen mühendislik ve önlisans başlangıç dilidir.

---

4. Derleyiciler (Compiler) ve Yorumlayıcılar (Interpreter)

Siz insan diline yakın bir C kodu yazdığınızda ("ekrana şunu yaz", "iki sayıyı topla"), bilgisayar bunu doğrudan çalıştıramaz. Bunu 0 ve 1'e çevirecek bir çevirmene ihtiyaç duyar. - **Derleyici (Compiler):** Yazılan binlerce satır kodun tamamını bir kerede okur, hatalar yoksa tamamını "Makine Diline" (.exe gibi dosyalara) çevirir. C dili bir derleyici kullanır. - **Yorumlayıcı (Interpreter):** Kodları okurken tek seferde dosyaya çevirmez, satır satır okur ve anında uygular. Python böyle çalışır.

---

5. Algoritmaya Hazırlık (Bir Sonraki Haftanın Habercisi)

Programlama, rastgele bilgisayara cümleler yazmak değildir, bir "Problem Çözme" sistematiğidir. Bir probleme çözüm bulmak için o çözümü adım adım formüle etmeliyiz. Önce düşünmeli, planlamalı sonra koda dökmeliyiz. İşte problemi çözmek için kurduğumuz bu adım adım yol haritasına ileride **"Algoritma"** diyeceğiz.

Örneğin, "Kek Yapma Algoritması": 1. Adım: Malzemeleri hazırla. 2. Adım: Yumurta ve şekeri çırp. 3. Adım: Un, yağ, süt ekle ve karıştır. 4. Adım: Önceden ısıtılmış kalıba dök. 5. Adım: 180 derece fırında pişir. 6. Adım: Çıkar ve soğut, dilimleyerek servis yap.

Bilgisayarlar da tıpkı bu adımlardaki gibi sırayla iş yapan aptal ama çok hızlı asistanlardır.

---

6. Ders Özeti ve Uygulama Alıştırmaları

Bu haftalık metnimizde, temel elektronik parçalar, işletim sistemi ile uygulama donanımı farkları, makine dili neden vardır ve neden programlama diline ihtiyaç duyuyoruz gibi başlıklara değindik.

Uygulama/Araştırma Soruları: 1. Şu an kullandığınız laboratuvar bilgisayarı veya şahsi bilgisayarınızın işlemcisi (CPU) ve RAM miktarı nedir? 2. İşletim sistemi olmadan bir bilgisayar çalışabilir mi? Tartışınız. 3. Derleyici ile yorumlayıcı arasındaki en büyük farkı bir benzetme yaparak açıklayınız. 4. Hayatınızda çok sık tekrarladığınız rutin bir sabah uyanma işlemini, adım adım yazarak kaba bir program planı (ön algoritma) hazırlayınız.

Bu içeriği en kısa sürede tekrar okuyarak, teknik kelimelerin sözlük anlamlarını not atmanız ileriki derslerimizde büyük kolaylık sağlayacaktır. Başarılar dileriz.

Hafta 2: Algoritma: Operatörler, Terimler ve Algoritma Tasarımı - I

1. Giriş ve Ön Hazırlık

Geçen hafta bilgisayarın çalışma prensiplerine ve programlamanın ne anlama geldiğine değindik. Bu hafta ise, kod yazmadan hemen önceki en kritik aşama olan "Algoritma" kavramına derinlemesine giriş yapıyoruz. Bilgisayarlar ne kadar hızlı olurlarsa olsunlar, onlara "ne yapacaklarını" adım adım söyleyecek bir mühendise (size) ihtiyaçları vardır.

Bu Haftanın Hedefleri: - Algoritma kavramının tanımını ve günlük hayattaki örneklerini kavrayabilmek. - Algoritmaların özellikleri ve tasarım prensiplerini anlamak. - Değişken (Variable) ve Sabit (Constant) gibi en temel yazılım terimlerini kavramak. - Matematiksel, ilişkisel ve mantıksal operatörlerin kullanım amaçlarını öğrenmek.

---

2. Algoritma Nedir?

Algoritma; belirli bir problemi çözmek ya da belirli bir amaca (sonuca) ulaşmak için tasarlanmış, **başlangıcı ve sonu belli olan**, adım adım kurallar (yönergeler) bütünüdür.

Algoritma bir bilgisayar kodu değildir, sadece problemin çözümünün insan dilindeki mantıksal dökümüdür. Doğru yazılmış bir algoritma, İngilizce, Türkçe veya matematikle ifade edilebilir; ardından Python, C, Java gibi diller aracılığıyla kodlanabilir.

2.1. Algoritmanın Temel Özellikleri

Geçerli ve işe yarar bir algoritmanın bazı evrensel kuralları olmalıdır: 1. **Giriş ve Çıkış:** Mutlaka bir başlangıcı (girdi) ve bir sonucu (çıktı) olmalıdır. 2. **Kesinlik (Netlik):** Adımlar çok net olmalı, yorum farkına dayalı kafa karışıklığı (belirsizlik) yaratmamalıdır. (Örn: "Biraz tuz ekle" yerine, "2 çay kaşığı tuz ekle" denilmelidir). 3. **Sonluluk:** Bir algoritma sonsuz döngüye(kısır döngü) girmeden makul bir deneme sonrasında kesinlikle sonlanmalıdır. 4. **Etkinlik:** Sorunu doğru ve gerektiği kadar adımda çözmelidir (ne çok kısa ne lüzumsuz derecede uzun).



3. Temel Terimler: Değişkenler ve Sabitler

Bilgisayar bir işlem yaparken verilere ihtiyaç duyar ve bu verileri geçici belleğinde (RAM üzerinde) depolar. Tıpkı bir mutfakta baharatların farklı kavanozlara konulması gibi, veriler de RAM'deki adreslerde tutulur.

3.1. Değişken (Variable) Nedir?

Çalışma zamanında (program akarken) içerisindeki değeri değişebilen, farklı veriler atanabilen RAM kutucuklarıdır. - Örnek: Bir oyundaki puan değeri (100 ile başlar, kurşun yedikçe 80'e, 50'ye düşer). Veya matematiksel bir işlemde dışarıdan okuduğunuz Sayi1 değişkeni.

Değişken İsimlendirme Kuralları: Programlamaya başlarken verilerimize isimler koyacağız (X, Sayi, Yas, Not). Bu aşamada değişkenlere isim verirken genel kurallar: - Sayı ile başlanmaz (5sayi olmaz, fakat sayi5 olur). - Özel karekter(_ alt tire hariç) ile başlanmaz ve özel karekter(_ alt tire hariç) içeremez. - Türkçe karakterler (ç, ş, ğ, ü, ö, ı) kullanılmamaya çalışılır. - Boşluk bırakılmaz (Ogrenci Notu yerine OgrenciNotu veya Ogrenci_Notu yazılır). - Değişken adı ilgili programlama diline ait anahtar kelimelerden oluşamaz.

3.2. Sabit (Constant) Nedir?

Program çalıştığı andan kapanana kadar içindeki değerinin hiç değişmeyeceği kesin olan verilere ait kutucuklardır. - Örnek: Matematik formülündeki "Pi (π) Sayısı" her zaman 3.14'tür program ortasında 4.2 olmaz. Ya da 1 haftadaki gün sayısı her zaman 7'dir sabittir.

---

4. Operatörler

Verileri işlemek, karşılaştırmak ve bir sonuca ulaşmak için kullandığımız sembollerdir. Operatörler, algoritmanın hesaplama kalbidir. Bizleri 3 temel gruba ayırıyoruz: Aritmetik, Karşılaştırma, Mantıksal.

4.1. Aritmetik (Matematiksel) Operatörler

Günlük hayattan bildiğiniz matematiksel işlemleri yaparlar: - **+ (Toplama)** - **- (Çıkarma)** - ***(Çarpma)** (Çarpı işareti olarak x değil yıldız(*) kullanırız). - **/ (Bölme)** - **% (Mod Alma):**

Programlamaya yeni giren bir kavramdır. "Bölümünden kalan" anlamındadır. Örneğin: `10 % 3 = 1`'dir. (10'u 3'e bölseniz kalan 1 olur). Algoritmalarda sayıların "tek mi çift mi" olduğunu anlamak için sıkça kullanılır.

4.2. İlişkisel (Karşılaştırma) Operatörleri

İki veriyi ölçüp tartar, sonuç olarak genellikle "Doğru (Evet)" veya "Yanlış (Hayır)" çıkartırlar. "Eğer bir durum varsa bunu yap" derken bu operatörler sorgulanır. - **> (Büyüktür):** `5 > 3` (Doğru) - **< (Küçüktür):** `4 < 2` (Yanlış) - **>= (Büyük Eşittir):** `Not >= 60` - **<= (Küçük Eşittir):** `Not <= 100` - **== (Eşittir):** Matematikteki tek =, atama yaparken; eğer iki sayının birebir aynı olup olmadığını "soruyorsak" iki kez eşittir == kullanılır. (`Sifre == "1234"` ise giriş yap.) - **!= (Eşit Değildir):** İki verinin farklı olduğunu ölçer. `Kullanici_Ad != ""` (Kullanıcı adı boş değilse işlemi yap.)

4.3. Mantıksal Operatörler

Birden fazla karşılaştırma veya şarta ihtiyacımız olduğunda şartları birbirine bağlar. - **VE (AND) Operatörü:** İstenen şartların **tümünün** doğru olmasını ister. Biri bile yanlışsa işlem iptal olur. (Örn: Burs alma şartı = "Not ortalaması 80'in üzerinde olacak" VE "Disiplin cezası almamış olacak") - **VEYA (OR) Operatörü:** İstenen şartlardan **en az birisinin** doğru olması yeterlidir. (Örn: Kayıt olma şartı = "Kredi Kartı ile ödeyecek" VEYA "Nakit ödeyecek". İkisinden biriyle işlem onaylanır). - **DEĞİL (NOT) Operatörü:** Mantığın tersini, tersine çevirmeyi kapsar. "Doğru" olanı "Yanlış", "Yanlış" olanı "Doğru" yapar.

---

5. Algoritma Yapıları Nelerdir?

Algoritmalar temelde yukarıdan aşağıya (1, 2, 3.. şeklinde) adım adım sırayla çalışırlar (Sıralı yapı). Ancak sorunlar ilerledikçe bu sıralı dizilişte "Seçim/Karar" yapısına girmek zorundayız.

Karar (Dallanma) Yapısı Nedir? Hayattaki bir yol ayrımı gibidir. Algoritma her yoldan gidemez, yukarıda gördüğümüz "İlişkisel Operatörler" ile sistem bir soru sorar, sonuca göre ya A yolundan ya da B yolundan akış devam eder. Örneğin yukarıda yazdığımız Vize-Final geçme algoritması: `Eğer (If) Puan >= 60 -> Geçecek Değilse (Else) -> Kalacak.`

---

6. Sözde Algoritma Tasarım Örnekleri

Örnek 1: İki ayrı notu alıp ortalama çıkaran algoritma:
Adım 1. Başla
Adım 2. Vize (V) notunu gir, Final (F) notunu gir
Adım 3. Ortalama = (V * 0.4) + (F * 0.6) hesabı yap
Adım 4. Bulunan "Ortalama" değerini Ekrana yaz.
Adım 5. Bitir.

Örnek 2: ATM'den Para Çekme Mantığı:
Adım 1. Başla
Adım 2. Kullanıcıdan kart şifresini girmesini iste.
Adım 3. **EĞER** girilen şifre 1234 eşit(==) ise 4'inci adıma geç. **DEĞİLSE** 6'ıncı adıma (Kartı iade et) git. (Karar yapısı)
Adım 4. Çekilecek tutarı gir.
Adım 5. Girilen tutar BankaHesabı'ndan Küçükse? Tutar kadar Hesabından düş DEĞİLSE yani Yetersizse "Bakiye Yetersiz" yaz.
Adım 6. Kartı İade et.
Adım 7. Bitir.

---

7. Hafta Özeti

Bu hafta algoritmalarda verilerin nasıl tutulacağını (Değişkenler), sabit olan değişmeyen verileri, iki sayı veya veriyi birbiriyle tarttığımız operatörleri ve mantık kapılarını öğrendik.

Alıştırmalar: 1. Günlük hayattan 2 tane Sabit veriye ve 2 tane Değişken veriye örnek veriniz. 2. `Sıcaklık == 100` ifadesi ile `Sıcaklık = 100` ifadesinin programlama mantığında çalışma amacı olarak farkı nedir? Araştırınız. 3. Elinizde 1'den 100'e kadar rakamlar var. Mod (%) operatörünü kullanarak o sayının çift bir sayı olup olmadığını tespit eden matematiksel sorgunuz nasıl olurdu? Kağıda yazınız.

Haftaya, operatörler üzerinden tasarımlar yapmaya (Algoritma Tasarımı - II) devam edeceğiz. Bol antrenman dileriz.

Hafta 3: Algoritma: Operatörler, Terimler, Algoritma Tasarımı - II

1. Giriş ve Ön Hazırlık

Bir önceki haftada algoritmaya giriş yaptık, temel operatörler ile değişkenleri tanımladık ve basit, sıralı, karar gerektiren problemler üzerine bir temel oluşturduk.

Bu haftanın konusu olan **Algoritma Tasarımı - II**'de, geçen dersteki Karar (If-Else) yapılarını biraz daha karmaşıklaştıracak, ilk kez "Döngü" (Tekrar - Loop) mantığının matematiksel temellerini kağıt üstünde kurgulayacağız. Bilgisayarların insanlardan asıl fark yarattığı özellikleri; sıkılmadan ve yorulmadan milyarlarca işlemi çok kısa bir sürede tekrarlayabilmeleridir. Bu hafta "Tekrar eden iş yapılarını" algoritmalara nasıl yedireceğimizi konuşacağız.

Hedefler: - İç-içe geçmiş karar (şart) yapılarını ayırt edebilmek. - Döngü/Tekrar algoritma bloğunun amacını kavramak. - Adım sayaçları (Sayac) mantığını algoritmaya uyarlamak. - Günlük ve matematiksel algoritma tasarımlarıyla analitik düşünme yeteneğimizi artırmak.

---

2. İç İçe Geçmiş Karar Yapıları

Karar yapısı, bir koşulun doğru ya da yanlış olmasına göre algoritmanın farklı adımları izlemesini sağlar. Önce tek koşullu basit kararları, ardından bir kararın içinde yeni bir kararın bulunduğu **iç içe geçmiş kararları** inceleyelim.

2.1. Basit Karar Yapısı

Bir koşulun sonucuna göre iki farklı işlemden birini seçmek, basit karar yapısıdır. Örneğin sınavdan geçme durumunu bulan algoritmada yalnızca notun 50 veya üzerinde olup olmadığına bakarız.

Örnek: Sınavdan Geçti mi?

1. Başla.
2. Öğrencinin sınav notunu alıp `Not` değişkenine koy.
3. **Eğer** `Not >= 50` ise ekrana "Sınavı Geçtiniz." yaz. **Değilse** ekrana "Sınavdan Kaldınız." yaz.
4. Bitir.

Bu örnekte tek bir soru sorulur. Koşul doğruysa bir mesaj, yanlışsa diğer mesaj gösterilir; ikinci bir koşul kontrol edilmez.

2.2. İç İçe Geçmiş Karar Yapısı

Bazı problemlerde ilk koşulun sonucu, başka bir koşulun kontrol edilip edilmeyeceğini belirler. İlk koşul sağlanıyorsa ikinci bir soru sorarız; sağlanmıyorsa bu soruya hiç geçmeyiz. Kararın içinde başka bir karar bulunduğu için bu yapıya **iç içe geçmiş karar yapısı** denir.

Örnek: Sınav Sonucuna Göre Başarı Durumu

Bu kez yalnızca geçme-kalma durumunu değil, sınavı geçen öğrencinin başarı düzeyini de bulalım. Önce geçme koşulunu kontrol eder, yalnızca geçen öğrenciler için ikinci koşula bakarız.

1. Başla.
2. Öğrencinin sınav notunu alıp `Not` değişkenine koy.

3. **Eğer** `Not >= 50` ise; - **Eğer** `Not >= 85` ise ekrana "Sınavı geçtiniz, başarı düzeyiniz yüksek." yaz. - **Değilse** ekrana "Sınavı geçtiniz." yaz.
4. **Değilse** ekrana "Sınavdan kaldınız." yaz.
5. Bitir.

İkinci koşul (`Not >= 85`), yalnızca ilk koşul (`Not >= 50`) doğru olduğunda kontrol edilir. Örneğin notu 40 olan bir öğrenci için algoritma doğrudan "Sınavdan kaldınız." mesajını verir; yüksek başarı koşulunu değerlendirmez.

Örnek: Ehliyet Başvurusu İçin Uygunluk

Ehliyet başvurusu için önce yaş koşulunu, yaş koşulu sağlanıyorsa sağlık raporunun bulunup bulunmadığını kontrol edelim.

1. Başla.
2. Kullanıcının yaşını alıp `Yas` değişkenine koy.
3. **Eğer** `Yas >= 18` ise; - Sağlık raporu olup olmadığını alıp `SaglikRaporuVarMi` değişkenine koy. - **Eğer** `SaglikRaporuVarMi` doğru ise ekrana "Sürücü kursuna kayıt olabilirsiniz." yaz. - **Değilse** ekrana "Yaş koşulunu sağlıyorsunuz ancak sağlık raporunuz eksik." yaz.
4. **Değilse** ekrana "18 yaşından küçük olduğunuz için başvuru yapamazsınız." yaz.
5. Bitir.

Burada ilk koşul yaşla ilgilidir. Kullanıcı 18 yaşından küçükse algoritma sağlık raporunu sormadan sonuç verir. Yaş koşulu sağlanırsa ikinci karar devreye girer. Böylece bir kararın sonucuna bağlı olarak başka bir kararın çalışmasını sağlamış oluruz.

Örnek: Sipariş Tutarına Göre Kargo Ücreti

Bir alışveriş sitesinde, sipariş tutarı belirli bir limite ulaşınca kargo ücretsiz olsun. Limitin altındaki siparişlerde ise standart veya hızlı kargo seçimine göre ücret belirlensin.

1. Başla.
2. Sipariş tutarını `SiparisTutari` değişkenine, kargo tercihini `HizliKargoMu` değişkenine al.
3. **Eğer** `SiparisTutari >= 1000` ise ekrana "Kargo ücretsiz." yaz.
4. **Değilse**; - **Eğer** `HizliKargoMu` doğru ise ekrana "Kargo ücreti 100 TL." yaz. - **Değilse** ekrana "Kargo ücreti 50 TL." yaz.
5. Bitir.

Bu örnekte kargo tercihi, yalnızca sipariş tutarı 1000 TL'nin altındaysa kontrol edilir. Böylece bir koşulun başka bir koşulun içinde yer aldığı üçüncü bir iç içe karar örneği görmüş olduk.

---

3. Sayıcılar (Counters) ve Toplayıcı Yapısı

Sıklıkla göreceğiniz terimlerden olan **Sayaç (Counter)**, algoritma içerisinde 1, 2, 3.. diye veya ikişer, üçer sistemli bir şekilde değeri artan (ya da azalan) bir değişkendir. Özellikle Döngülerde ne kadar tekrar edeceğimizi bulmak için sayaç kurarız. - Sayacı artırmak: `Sayac = Sayac + 1` (Matematikte bu formül yanlış görünse de, programlamada anlamı şudur: Sayaç kutusunun eski değerinin üstüne 1 ekle ve kendi içine geri kayıt et.)

Aynı durum hesaplamalarda **Toplayıcılar** için geçerlidir: - `Toplam = Toplam + Fiyat` (Kasadan her okutulan fiyatta, ana tutarın üstüne eklene eklene yeni bakiye oluşur.)

---

4. Döngü (Tekrar - Loop) Mantığına Giriş

Bir işlemin belirli bir amaca veya koşula ulaşana kadar tekrar edilmesine **Döngü (Loop)** denir.

Bir döngünün temelinde üç adım vardır: başlangıç değerini belirlemek, tekrarlanacak işlemi yapmak ve her turda değeri güncelleyip devam koşulunu kontrol etmek. Koşul sağlandığı sürece işlem tekrarlanır; sağlanmadığında döngü biter.

Örnek 1: Afişleri Numaralandırma

Bilgisayarın ekrana "Afiş 1", "Afiş 2" ve bu şekilde "Afiş 100" yazmasını isteyelim. Döngü kullanmazsak her afiş için ayrı bir adım yazmamız gerekir. Sayaç kullanarak bu tekrarları kısa bir algoritmayla yapabiliriz.

1. Başla.
2. Sayac = 1 olarak başlangıç değerini ata.
3. Ekrana "Afiş " metnini ve Sayac değerini yaz.
4. Sayac = Sayac + 1 işlemini yap.
5. **Eğer** Sayac <= 100 ise Adım 3'e geri dön.
6. **Değilse** Bitir.

Sayaç her turda bir arttığı için algoritma afişleri 1'den 100'e kadar sırasıyla yazar ve 100'ü geçince durur.

Örnek 2: 1'den 3'e Kadar Sayma

Sayaç kullanarak ekrana 1, 2 ve 3 sayılarını yazdıralım.

1. Başla.
2. Sayac = 1 olarak başlangıç değerini ata.
3. Sayac değerini ekrana yaz.
4. Sayac = Sayac + 1 işlemini yap.
5. **Eğer** Sayac <= 3 ise Adım 3'e geri dön.
6. **Değilse** Bitir.

Örnek 3: Geri Sayım

Bir başlangıç sayısından 1'e kadar geriye doğru sayıp ekrana yazdıralım. Bu örnekte sayaç artmak yerine her turda bir azalır.

1. Başla.
2. Sayac = 5 olarak başlangıç değerini ata.
3. Sayac değerini ekrana yaz.
4. Sayac = Sayac - 1 işlemini yap.
5. **Eğer** Sayac >= 1 ise Adım 3'e geri dön.
6. **Değilse** Bitir.

Ekrana sırasıyla 5, 4, 3, 2 ve 1 yazılır. Sayaç 0 olduğunda koşul yanlış olur ve döngü sona erer.

---

5. Algoritma Tasarımı - II : Örnek Senaryolar

Aşağıda hem iç içe şart hem de sayaç/döngü algoritma tasarımlarına yer verilmiştir. Metin olarak tasarım yapmaya iyice adapte olunuz, zira haftaya bu metinleri sembollerle "Akış Diyagramlarına" dökeceğiz.

Örnek 1: Sayının İşaretini Bulma

Soru: Dışarıdan girilen bir sayının Pozitif mi, Negatif mi yoksa Sıfır (0) mı olduğunu bulan algoritmayı kurun. (Burada 3 ihtimal var, tek soru (Evet/Hayır) yetmez, İç İçe sorulmalıdır.)

1. Başla.
2. "Lütfen Bir Sayı Girin:" mesajını göster.
3. Kullanıcının bilgisini alıp Sayi değişkenine koy.

4. **Eğer** `Sayi == 0` ise; - Doğru ise ekrana "Sayınız Sıfırdır." yaz. - DEĞİLSE;
    - **Eğer** `Sayi > 0` ise;
    - Doğru ise ekrana "Sayınız Pozitiftir." yaz.
    - DEĞİLSE (Yani Ne 0, ne de Büyük değilse zaten Negatiftir) "Sayınız Negatiftir" yaz.
5. Bitir.

Örnek 2: Döngü ve Toplayıcı Mantığı. (1'den N'ye Kadar Olan Sayıları Toplamak)

Soru: Kullanıcı sistem bir tavan sayı girecek (Diyelim ki 5 girdi). Sistemin görevi: (1+2+3+4+5) bu işlemi otomatik hesaplayıp sonucu bulacak.

1. Başla.
2. Dışarıdan `LimitSayı` değişkenini al. (Örn 5 olsun).
3. Hafızada `Sayac = 1` ve `Toplam = 0` adında iki tane başlangıç değişkeni kutusu ata.
4. `Toplam = Toplam + Sayac` işlemini yap (Sıfırın üstüne 1 eklendi, artık toplam 1).
5. `Sayac = Sayac + 1` işlemini yap (Sayac'ı bir arttırdık. 2 oldu).
6. **Eğer :** `Sayac <= LimitSayı` ise Demekki daha bitmemiş işimiz, **Adım 4'e Geri Dön**. (Döngü). - **DEĞİLSE** Ekrana nihai hesabı bulduğunuz `Toplam` değişkenini yazdır.
7. Bitir.

---

6. Haftalık Özet

Bu hafta algoritmalarımıza yön verirken hem sayaç mekanizmalarını öğrendik hem de "tekrar işlemlerine" bilgisayarları nasıl mecbur bıraktığımızın (Adım X'e geri dön taktiğinin) taslağını oluşturduuk. Bu yapıları mantıksal olarak kavradığınız an, herhangi bir yazılıme kodunu okuduğunuzda (Aaa burada koşul açmış kodları geri fırlatıyor) şeklindeki sistematiği çok rahat göreceğinizi hissedebilirsiniz.

Alıştırma Soruları: 1. Bilgisayardan çarpım tablosunda sadece 5'leri (5x1=5, 5x2=10, 5x9=45.. gibi) bir döngü kullanarak tasarlayıp ekrana yazdırmak isterseniz nasıl bir Sözel Algoritma adımı tasarlar, hangi adımlara döngüyü fırlatırdınız? Kendi kağıdınıza, yukarıdaki örneklere benzer yazınız. 2. "Mantıksal Operatörler" başlığında geçen VEYA (OR) kullanılarak, "Ürün iadesi için ürünün ya fişinin olması VEYA kullanıcı kartıyla alınmış olması"na bakan iki durumlu bir sözel Karar Algoritması tasarlayınız.

Hafta 4: Akış Diyagramları: Semboller, Analiz ve Tasarım - I

1. Giriş ve Ön Hazırlık

Daha önceki haftalarımızda operatörleri, algoritma kavramını ve bir problemi sözel (metin) olarak nasıl adım adım çözeceğimizi (sözde kod tasarımlarını) öğrendik. Ancak sadece yazıyla ifade edilen uzun algoritmaları takip etmek, özellikle karmaşık problemlerde kafa karıştırıcı olabilir.

İşte tam bu noktada, yazıyla anlattığımız algoritma çözümlerimizi **görsel şekiller** vasıtasıyla geometrik sembollere aktarırız. Bu görsel şablonlara **"Akış Diyagramı" (Flowchart)** adını veriyoruz.

Önlisans programımızda temel hedefimiz, kod yazma işlemine geçmeden önce algoritmik düşünce becerinizi görsel olarak zihninize kazımaktır.

---

2. Neden Akış Diyagramı Kullanılır?

- **Görsellik:** İnsan beyni, karmaşık metinlerden ziyade diagram ve şekilleri saniyeler içinde anlayıp yorumlayabilir.
- **Hatayı Çabuk Görme:** Sürecin hangi yöne aktığı, şartların nerede ikiye ayrıldığı oklara bakarak saniyesinde tespit edilebilir. Mantık hatası (Logic Error) ihtimali çok azalır.
- **Standart Dil:** Siz Türkiye'de bir akış diyagramı çizerseniz, bunu İngilizce bile bilmeyen Çin'deki bir yazılımcı dahi sembollerin evrensel olması sayesinde kolaylıkla okuyup kodlayabilir.

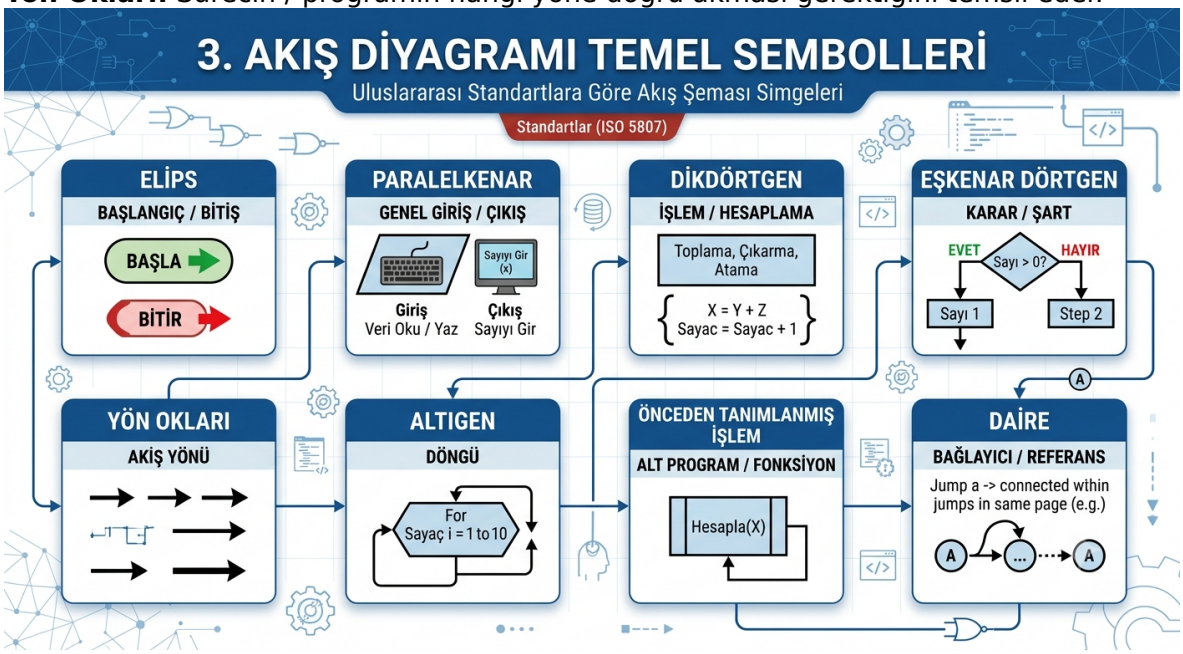
---

3. Akış Diyagramı Temel Sembolleri

Diyagramlarımız kafamıza göre çizdiğimiz şekiller değildir; uluslararası standartları vardır. İşte en temel basit seviye sembollerimiz:

1. **Elips (Oval):** Programın başladığı ve bittiği noktaları gösterir. Algoritma her zaman bir "Başla" elipsi ile başlar, işler bitince "Bitir" elipsi ile sonlanır.
2. **Paralelkenar:** Dışarıdan bilgi okumak (klavyeden giriş yapmak) veya sadece basit ekrana yansıtma (çıkış) durumlarında kullanılır (Genel giriş / çıkış birimidir).
3. **Dikdörtgen:** İşlem, atama veya hesaplama kutusudur. Tüm matematiksel işlemler (toplama, çıkarma) burada yapılır.
4. **Eşkenar Dörtgen (Karar / Şart):** Mantıksal karşılaştırmalar içerir. (Eğer sayı sıfırdan büyükse sağa git, küçükse aşağı git gibi). İçinden "Evet" ve "Hayır" olmak üzere iki ok çıkar.
5. **Dalgalı Dikdörtgen (Yazdırma):** Ekrana çıktı olarak gösterilecek ifadeyi temsil eder.
6. **Altıgen (Döngü):** Döngüyü ifade eder.

7. **Yön Okları:** Sürecin / programın hangi yöne doğru akması gerektiğini temsil eder.



---

4. Akış Diyagramı Tasarım Örnekleri

Aşağıda, 3 farklı seviyede algoritma problemi ve bunların Meramid yardımıyla oluşturulmuş standart akış diyagramlarını bulacaksınız.

ÖRNEK 1: İki Sayının Toplamını Bulan Program

Sözel Algoritma: 1. Başla 2. Bellekten birinci sayıyı iste (Sayı1). 3. Bellekten ikinci sayıyı iste (Sayı2). 4. İkisini Topla ve "Toplam" olarak sakla (Toplam = Sayı1 + Sayı2). 5. "Toplam" değerini ekranda göster. 6. Bitir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/1. Sayıyı Giriniz: Sayi1/]
    B --> C[/2. Sayıyı Giriniz: Sayi2/]
    C --> D[Toplam = Sayi1 + Sayi2]
    D --> E@{ shape: doc, label: "Toplam" }
    E --> F([Bitir])
```

Açıklama: Başla ve Bitir elipslerle belirtildi. Dışarıdan Sayi1 ve Sayi2 alınırken Giriş/Çıkış sembolü (burada paralelogram niyetine /.../ kullanıldı) uygulandı. İşlem (toplama işlemi) standart dikdörtgen olarak tasarlandı.

ÖRNEK 2: Girilen Sayının Tek mi, Çift mi Olduğunu Bulan Program

Bu örnekte işin içine bir "KARAR (ŞART)" işlemi, yani Eşkenar Dörtgen giriyoruz. Sisteme bir sayı verilecek, bu sayı 2'ye tam bölünüyorsa "Çift", bölünmüyorsa "Tek" denilecek.

Sözel Algoritma: 1. Başla. 2. Kullanıcıdan bir sayı girilmesini iste. 3. Eğer (Sayı % 2 == 0) ise "Sayınız Çifttir" yaz. 4. Değilse ekrana "Sayınız Tektir" yaz. 5. Bitir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/Bir Sayı Giriniz/]
    B --> C{Sayı%2==0}
```

```
    C -- EVET --> D@{ shape: doc, label: "Sayı Çifttir" }
    C -- HAYIR --> E@{ shape: doc, label: "Sayı tektir" }

    D --> F([Bitir])
    E --> F([Bitir])
```

Açıklama: Algoritmamız ilk defa dallanma işlemi ("branching") yaptı. Karar kutucuğumuz soruyu sordu, sağa veya sola doğru EVET/HAYIR yanıtlarına göre gidişat şekillendi, ama sonuçta iki ok da programın kapanışı olan Bitir'de buluştu.

ÖRNEK 3: Vize ve Final Notu İle Geçme Durumu Hesaplama

Üniversitelerde yaygın kullanılan dersten geçme sisteminin algoritmasıdır. Vizenin %40'ı ile finalin %60'ı alınır, toplamı (ortalama) eğer 60 ve üzerinde ise Geçti, 60'tan küçükse Kaldı yazılır.

Sözel Algoritma: 1. Başla. 2. Vize notunu gir. 3. Final notunu gir. 4. Ortalama = (Vize * 0.4) + (Final * 0.6) 5. Eğer Ortalama >= 60 ise "Dersten Geçtin" yaz. 7. Değilse "Dersten Kaldın" yaz. 8. Bitir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/Vize Notunu Gir/]
    B --> C[/Final Notunu Gir/]
    C --> D[Ortalama = Vize*0.4 + Final*0.6]
    D --> E{Ortalama >= 60 mı?}

    E -- EVET --> F@{ shape: doc, label: "Dersten geçtiniz" }
    E -- HAYIR --> G@{ shape: doc, label: "Dersten kaldınız" }

    F --> H([Bitir])
    G --> H([Bitir])
```

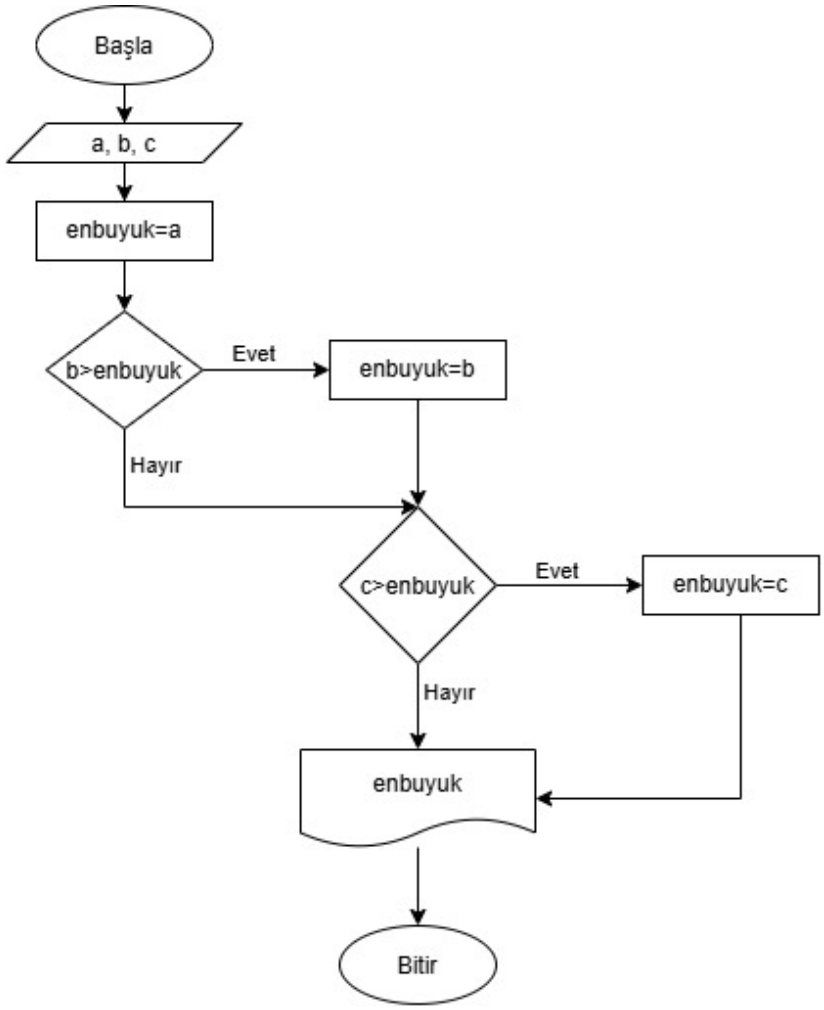
Açıklama: Burada hem bir hesaplama (Dikdörtgen İşlem Alanı) kullandık hem de bir algoritmik şart (Karar - Eşkenar Dörtgen) kullandık. Üçüncü örneğimiz, günlük hayattaki otomasyonların bilgisayarın mantığında nasıl işlediğine en güzel yatkınlığı sağlar.

ÖRNEK 4: Klavyeden Girilen Üç Sayının En Büyüğünü Bulan Program

Bu örnekte klavyeden alınan üç sayı karşılaştırılır ve en büyük olan sayı bulunur. Önce ilk sayı en büyük kabul edilir; ardından ikinci ve üçüncü sayılar sırayla bu değerle karşılaştırılır.

Akış Diyagramı Şeması:

a	b	c
5	2	1
1	5	2
2	1	5
5	4	5



Açıklama: a, b ve c değerleri alındıktan sonra enbuyuk değişkenine önce a atanır. b ve ardından c, o anki en büyük değerle karşılaştırılır; daha büyük olan varsa enbuyuk güncellenir ve sonuç gösterilir.

ÖRNEK 5: Dikdörtgenin Alanını ve Çevresini Hesaplayan Program

Bu akış diyagramında kullanıcıdan dikdörtgenin kısa ve uzun kenar uzunlukları alınır. Alan `kısa kenar × uzun kenar`, çevre ise `2 × (kısa kenar + uzun kenar)` formülüyle hesaplanıp ekranda gösterilir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/kısaKenar gir/]
    B --> C[/uzunKenar gir/]
    C --> D[Alan = kısaKenar * uzunKenar]
    D --> E["Çevre = 2 * (kısaKenar + uzunKenar)"]
    E --> F@{ shape: doc, label: "Alan ve çevreyi göster" }
    F --> G([Bitir])
```

ÖRNEK 6: Dairenin Alanını ve Çevresini Hesaplayan Program

Bu örnekte kullanıcıdan dairenin yarıçapı alınır. Alan `π × r × r`, çevre ise `2 × π × r` formülleriyle hesaplanır. Hesaplamalarda π yaklaşık `3,14` olarak kullanılır.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/r gir/]
```

```
    B --> C[Alan = 3.14 * r * r]
    C --> D[Çevre = 2 * 3.14 * r]
    D --> E@{ shape: doc, label: "Alan, Çevre" }
    E --> F([Bitir])
```

ÖRNEK 7: Sayının Pozitif, Negatif veya Sıfır Olduğunu Bulan Program

Bu örnekte kullanıcıdan bir sayı alınır. Önce sayının sıfırdan büyük olup olmadığı kontrol edilir; değilse sıfırdan küçük olup olmadığına bakılır. İki koşul da sağlanmıyorsa sayı sıfırdır.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/x/]
    B --> C{x > 0 mı?}
    C -- Evet --> D@{ shape: doc, label: "x pozitiftir" }
    C -- Hayır --> E{x < 0 mı?}
    E -- Evet --> F@{ shape: doc, label: "x negatiftir" }
    E -- Hayır --> G@{ shape: doc, label: "x sıfırdır" }
    D --> H([Bitir])
    F --> H
    G --> H
```

ÖRNEK 8: Yaşa Göre Reşitlik Durumunu Bulan Program

Kullanıcıdan yaşı alınır ve 18 yaşını doldurup doldurmadığı karar yapısıyla kontrol edilir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/Yaşı gir/]
    B --> C{Yaş >= 18 mi?}
    C -- Evet --> D@{ shape: doc, label: "Reşitsiniz" }
    C -- Hayır --> E@{ shape: doc, label: "Reşit değilsiniz" }
    D --> F([Bitir])
    E --> F
```

ÖRNEK 9: Üç Kenarla Üçgen Oluşup Oluşmadığını Bulan Program

Kullanıcıdan üç kenar uzunluğu alınır. Kenarlar pozitifse üçgen eşitsizliği kontrol edilir: Herhangi iki kenarın toplamı, üçüncü kenardan büyük olmalıdır.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/a, b ve c kenar uzunluklarını gir/]
    B --> C{a > 0 ve b > 0 ve c > 0 mı?}
    C -- Hayır --> D@{ shape: doc, label: "Kenar uzunlukları pozitif olmalı" }
    C -- Evet --> E{a+b > c ve a+c > b ve b+c > a mı?}
    E -- Evet --> F@{ shape: doc, label: "Üçgen oluşur" }
    E -- Hayır --> G@{ shape: doc, label: "Üçgen oluşmaz" }
    D --> H([Bitir])
    F --> H
    G --> H
```

ÖRNEK 10: 1'den 5'e Kadar Sayıları Yazdırma

Soru: 1'den başlayarak 5'e kadar olan sayıları ekrana yazdıran akış diyagramını oluşturunuz. Karar yapısı, sayacın 5'e ulaşıp ulaşmadığını kontrol ederek döngüyü sürdürür veya bitirir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[i = 1]
    B --> C{i <= 5 mi?}
```

```
    C -- Evet --> D@{ shape: doc, label: "i değerini yazdır" }
    D --> E[i = i + 1]
    E --> C
    C -- Hayır --> F([Bitir])
```

ÖRNEK 11: 1'den 10'a Kadar Sayıların Toplamını Bulma

Soru: 1'den 10'a kadar olan tam sayıların toplamını bulan akış diyagramını oluşturunuz. Her turda sayaç toplama eklenir ve sayaç artırılır; sayaç 10'u geçince toplam ekranda gösterilir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[i = 1, toplam = 0]
    B --> C{i <= 10 mu?}
    C -- Evet --> D[toplam = toplam + i]
    D --> E[i = i + 1]
    E --> C
    C -- Hayır --> F@{ shape: doc, label: "toplamı göster" }
    F --> G([Bitir])
```

---

5. Haftalık Özet ve Alıştırma

Programlama yolculuğunuzda, problemleri algoritmaya çevirmeyi kolaylaştırmak amacıyla akış diyagramı modelleme dilini öğrendik. Doğru çizilmiş bir akış şeması, **C dili** başta olmak üzere her türlü programlama diline %99 doğrulukta bir çeviri demektir. O şema elinizdeyse, sadece çevirmek kalır.

Laboratuvar / Bireysel Çalışma Soruları: 1. Klavyeden kullanıcının girdiği bir doğum tarihine göre kullanıcının "Yaşını" hesaplayan (Günümüz Yılı - Doğum Yılı) formülü yazan ve akış diyagramını kendi defterinize semboller ile çizen bir algoritma tasarlayınız. 2. Akış sembolleri içindeki "İşlem / Atama (Dikdörtgen)" ile "Karar / Seçim (Eşkenar Dörtgen)" amaç olarak birbirinden neden ayrılmıştır?

Önümüzdeki hafta, bu diagramlarda birden çok durum (iç içe kontroller) kısmını daha karmaşık senaryolarla (Akış Diyagramları II) işleyecek ve zihinsel hazırlığımızı tamamlayacağız.

Hafta 5: Akış Diyagramları: Semboller, Analiz ve Tasarım - II

1. Giriş ve Ön Hazırlık

Geçtiğimiz hafta akış diyagramı dünyasına ayak basmış, standart sembollerimizle (Elips, Eşkenar Dörtgen, Dikdörtgen) basit karar ve sıralı kodlamaların diyagram şemalarını incelemiştik.

Hafta 3'te "Algoritma 2" konusunda Döngü (Tekrar) mekanizmalarını da sözel metin olarak çalıştık ama işin resmi haline dökmedik. O yüzden, bu 5. haftamızda özellikle Algoritmadaki en zorlayıcı olgu olan **Döngü ve Karmaşık İç İçe Karar** yapılarını "Akış Diyagramlarıyla" görselliğe aktarmaya odaklanacağız.

Hedefler: - Oku alıp geriye göndererek "Döngü (Loop)" akışını diyagramlaştırmak. - Sayaç (Counter) ve Toplam mekanizmalarının geometrik yansımalarını öğrenmek. - İki veya daha fazla karar (şart/Eşkenar Dörtgen) bağlanan durumlarda çoklu yolları idare etmek.

---

2. Döngüleri Diyagrama Dökmek Üzerine Ön Bilgi

Sıralı algoritmaların diyagramlarındaki oklar hep aşağı akar veya sağa / sola seker en nihayetinde de "Bitir" topunda birleşirdi. **Döngülü** akış şemalarında da karar oklarından biri (Genelde "Evet, daha döngü bitmedi" okudur bu), sürecin aşağıya akan mekanizmasına veda eder, çizgi yukarı doğru seyrederek eski bir dikdörtgenin başına tekrar bağlanır.

Döngünün sonsuza girmemesi için daima araya bir "Sayaç" koymalı ve her turda onu 1 attırmak üzere işlem satırını araya sıkıştırmalıyız. Karar şartı her turda bu sayacın son limiti aşıp aşmadığını test edip "Aşmadıysa Devam Et", "Aştıysa Döngüden Çık ve Bitir" denmelidir.

---

3. Akış Diyagramı Tasarım Örnekleri (Döngü / İleri Kararlar)

İlk olarak döngü ve sayaç mantığını gösteren 3 örnek, ardından `for` döngüsünün akış diyagramındaki altıgenlerle gösterildiği 5 basit örnek ele alınmıştır.

ÖRNEK 1: Ekrana 5 Kere "Merhaba Dünya" Yazdıran Döngü Algoritması

Yazılım serüvenlerinde "10 kere tekrar et" komutunu diyagramla ifade edeceğiz. Klasik mantıkla art arda 5 tane çıktı diktörtgeni çizebilecek olmamıza rağmen, ya 100.000 kere istenirse diye döngü sistemini kullanacağız.

Algoritma Adımları: 1. Başla 2. Adim isimli değişkene 1 ataması yap (Adım sayacı) 3. Ekrana "Merhaba Dünya" çıkışını ver. 4. Adim = Adim + 1 yap 5. Eğer Adim <= 5 ise Adım 3'e GİT. (Büyükse Adım 6'ya devam et) 6. Bitir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[Adim = 1]
    B --> C@{ shape: doc, label: "Merhaba Dünya" }
    C --> D[Adim = Adim + 1]
    D --> E{Adim <= 5 mi?}

    E -- EVET, Sayi 5'in altinda veya 5'te --> C
    E -- HAYIR, 6 oldu --> F([Bitir])
```

Açıklama: Dikkat ettiyseniz, E harfinden çıkan "Evet" cevabı, algoritmayı bitirmiyor ve yukarı, C'nin bulunduğu noktaya tekrar bir Ok yolluyor. İşte kodlama dünyasında döngü denilen kısım bu yukarı giden optur; şart bitene kadar burada bir hortum oluşacaktır.

ÖRNEK 2: Kullanıcı Tarafından Belirlenen N Sayısına Kadar Toplam Bulma (1'den N'ye)

Diyelim ki programı çalıştıran kişi sisteme 4 sayısını yazdı. Program 1+2+3+4 hesaplayacak ve bulduğu 10 sayısını çıktı olarak kullanıcıya gösterecek. Limitimiz klavyeden belirleniyor.

Algoritma Adımları: 1. Başla. 2. Dışarıdan hedeflenen limit olan sayı girişini (`N`) iste. 3. Arka plan hafızasında `Tur=1` ve `Toplam=0` değişkenlerini yarat. 4. `Toplam = Toplam + Tur` (0 olan toplama, turun değerini aktar). 5. `Tur = Tur + 1` (Turu bir sonraki aşamaya çek, yani 2 yap vb.) 6. Püf Noktası – Soru Sor: `Tur <= N` değerine hala uygun mu? Doğruysa 4. adıma geri fırlat. 7. Eğer şart ihlal edildiyse, ekrana `Toplamı` gönder. 8. Bitir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[/Lütfen N Sınır Sayısını Gir/]
    B --> C[Tur = 1, Toplam = 0]
    C --> D[Toplam = Toplam + Tur]
    D --> E[Tur = Tur + 1]
    E --> F{Tur <= N mi?}

    F -- EVET, Döngü Devam --> D
    F -- HAYIR, Limiti Aştık --> G@{ shape: doc, label: "Toplam" }

    G --> H([Bitir])
```

Açıklama: Tur değeri limit sayısını her bir arttırdığında D e bağlanan koca bir tur çemberine girer. Limit aşıldığında sistem çemberi kollarından koparark Hayır okunu seçer ekrana skoru yazar. Döngü ile Karar mekanizmasının sanatına harika bir taslaktır.

ÖRNEK 3: Kullanıcı Pozitif veya Sıfır Girdiği Sürece Girdiği Sayıları Toplayan Sistemin Akış Şeması

Bu örnek, yazılımda "Sonsuz Döngüye veya Şarta Bağlı Döngüye (While)" örnektir. Limiti önceden bilmeyiz, tek derdi; "Kullanıcı önüne gelen ekrana Eksi (-) sayılar (-4 gibi) girmediği SÜRECE" işlemi tekrarlatıp yazılan sayıları kumbaraya atsın, kullanıcı negatif girer girmez o sayıyı almayıp eskilerin tüm toplamını verip işlemi kırsın (programı bitirsin).

Algoritma Adımları: 1. Başla. 2. Hafızada Toplam değerine sıfır ver (`Toplam=0`). 3. Dışarıdan Sayı İste. 4. Girilen `Sayi >= 0` mı sorusunu Karara bağla. 5. Eğer Sıfır'dan büyük/eşitse (Şart Tutuyorsa): `Toplam = Toplam + Girilen Sayi` formülünü yap. Ardından yine Adım 3'e çık, yeni bir sayı girmesini iste (Yine Karara gelsin vb). 6. Eğer Sıfır'dan değil, Eksiyse (negatif-şart tutmuyorsa): O koldan kop, Ekrana "Su Ana Kadar Girilen Toplamınız= Toplam", yazdır. 7. Bitir.

Akış Diyagramı Şeması:

```
graph TD
    A([Başla]) --> B[Toplam = 0]

    B --> C[/Kullanıcıdan Bir Sayı Girilmesini İste/]

    C --> D{Girilen Sayı >= 0 mı?}

    D -- EVET, Kapsamda --> E[Toplam = Toplam + Sayi]
    E --> C
```

```
    D -- HAYIR, Negatif Bir Rakam Girdi --> F@{ shape: doc, label: "Toplam" }
    F --> G([Bitir])
```

Açıklama: Bu döngümüzde net bir 'Sayac = Sayac+1' durumu yok, şart kullanıcıya bağlandı. Kullanıcı klavyeden -5 yazmadığı müddetçe, sistem sonsuza kadar onu (C) aşamasıyla (E) aşaması arasında defalarca çalışabilecek bir kafesin içerisine attı. Yazılımların interaktif olarak insana hizmet etmesini saplayan kilit prensip buradaki diyagramdır.

For Döngüsünü Altıgenlerle Gösterme

Akış diyagramında altıgen (hazırlık sembolü) `for` döngüsünün tamamını tek kutuda gösterir: `başlangıç; şart; artış`. Ayrıca karar eşkenar dörtgeni veya sayaç artırma kutusu çizilmez. Şart doğruysa döngü gövdesi çalışır ve ok altıgene geri döner (sayaç otomatik artar); şart yanlışsa döngüden çıkılır.

ÖRNEK 4: 1'den 5'e Kadar Sayıları Yazdırma

Sayaç 1'den başlar. 5'i aşmadığı sürece ekrana yazdırılır ve bir artırılır.

Algoritma Adımları: 1. Başla. 2. `i = 1` yap (sayaç başlangıcı). 3. `i <= 5` şartını kontrol et. Şart yanlışsa Adım 6'ya git. 4. Ekrana `i` değerini yazdır. 5. `i = i + 1` yap ve Adım 3'e dön. 6. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B{{"i = 1; i <= 5; i++"}}
    B -- Şart doğru --> C@{ shape: doc, label: "i değerini yazdır" }
    C --> B
    B -- Şart yanlış --> D([Bitir])
```

ÖRNEK 5: 1'den 5'e Kadar Sayıların Toplamını Bulma

Her turda sayaç toplam değişkenine eklenir. Döngü bitince toplam ekrana yazdırılır.

Algoritma Adımları: 1. Başla. 2. `toplam = 0` yap. 3. `i = 1` yap (sayaç başlangıcı). 4. `i <= 5` şartını kontrol et. Şart yanlışsa Adım 7'ye git. 5. `toplam = toplam + i` yap. 6. `i = i + 1` yap ve Adım 4'e dön. 7. Ekrana `toplam` değerini yazdır. 8. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[toplam = 0]
    B --> C{{"i = 1; i <= 5; i++"}}
    C -- Şart doğru --> D[toplam = toplam + i]
    D --> C
    C -- Şart yanlış --> E@{ shape: doc, label: "toplam değerini yazdır" }
    E --> F([Bitir])
```

ÖRNEK 6: Tek Sayıları Yazdırma

1'den başlayıp sayacı her turda 2 artırarak 9'a kadar olan tek sayılar yazdırılır.

Algoritma Adımları: 1. Başla. 2. `sayi = 1` yap. 3. `sayi <= 9` şartını kontrol et. Şart yanlışsa Adım 6'ya git. 4. Ekrana `sayi` değerini yazdır. 5. `sayi = sayi + 2` yap ve Adım 3'e dön. 6. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B{{"sayi = 1; sayi <= 9; sayi += 2"}}
    B -- Şart doğru --> C@{ shape: doc, label: "sayi değerini yazdır" }
```

```
    C --> B
    B -- Şart yanlış --> D([Bitir])
```

ÖRNEK 7: 3'ün Çarpım Tablosunu Yazdırma

Sayaç 1'den 5'e kadar ilerler; her turda 3 ile sayacın çarpımı yazdırılır.

Algoritma Adımları: 1. Başla. 2. `i = 1` yap (sayaç başlangıcı). 3. `i <= 5` şartını kontrol et. Şart yanlışsa Adım 6'ya git. 4. Ekrana `3 * i` değerini yazdır. 5. `i = i + 1` yap ve Adım 3'e dön. 6. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B{{"i = 1; i <= 5; i++"}}
    B -- Şart doğru --> C@{ shape: doc, label: "3 * i değerini yazdır" }
    C --> B
    B -- Şart yanlış --> D([Bitir])
```

ÖRNEK 8: 5 Sayısının Faktöriyelini Hesaplama

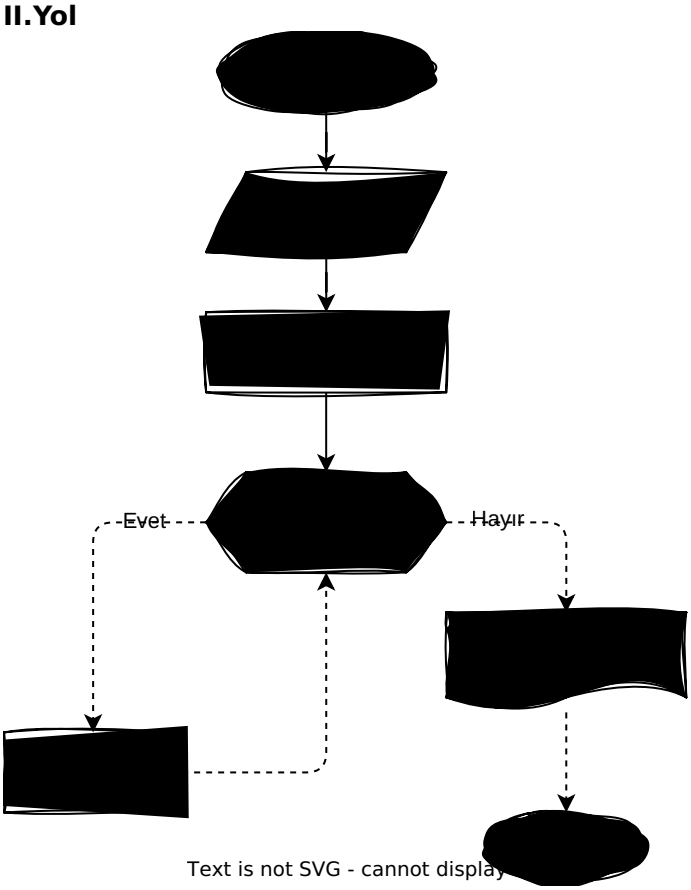
Sayaç 1'den 5'e kadar ilerler ve her turda çarpım sonucuna eklenir. Döngü sonunda `5! = 120` bulunur.

Algoritma Adımları: 1. Başla. 2. `faktoriyel = 1` yap. 3. `i = 1` yap (sayaç başlangıcı). 4. `i <= 5` şartını kontrol et. Şart yanlışsa Adım 7'ye git. 5. `faktoriyel = faktoriyel * i` yap. 6. `i = i + 1` yap ve Adım 4'e dön. 7. Ekrana `faktoriyel` değerini yazdır. 8. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[faktoriyel = 1]
    B --> C{{"i = 1; i <= 5; i++"}}
    C -- Şart doğru --> D[faktoriyel = faktoriyel * i]
    D --> C
    C -- Şart yanlış --> E@{ shape: doc, label: "faktoriyel değerini yazdır" }
    E --> F([Bitir])
```

II.Yol



Orta Seviye Döngü Soruları (For / While)

Çizim kuralları: Döngü şartı (`for` için başlangıç; şart; artış) altıgenin içinde yazılır, ayrıca eşkenar dörtgen çizilmez. Eşkenar dörtgen yalnızca döngü içindeki gerçek `if` kararlarında kullanılır. Ekrana yazdırma doc şekliyle, klavyeden okuma paralelkenarla gösterilir.

SORU 1: 1'den 10'a Kadar Çift Sayıların Toplamı (For + Karar)

1'den 10'a kadar sayıları gezin, yalnızca çift olanları toplayın ve toplamı yazdırın. (Sonuç: 30)

Algoritma Adımları: 1. Başla. 2. `toplam = 0` yap. 3. `i = 0` yap (sayaç başlangıcı). 4. `i <= 10` şartını kontrol et. Şart yanlışsa Adım 7'ye git. 5. `toplam = toplam + i` yap. 6. Her döngü turundan sonra `i` değerini 2 artır ve Adım 4'e dön. 7. Ekrana `toplam` değerini yazdır. 8. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[toplam = 0]
    B --> C{{"i = 0; i <= 10; i+=2"}}
    C -- Şart doğru --> D[toplam = toplam + i]
    D --> C
    C -- Şart yanlış --> E@{ shape: doc, label: "toplam değerini yazdır" }
    E --> F([Bitir])
```

SORU 2: N'e Kadar Sayıların Karelerini Yazdırma (For)

Kullanıcıdan N değerini alın. 1'den N'e kadar her sayının karesini ekrana yazdırın. (N = 4 için: 1, 4, 9, 16)

Algoritma Adımları: 1. Başla. 2. Kullanıcıdan `N` sayısını al. 3. `i = 1` yap (sayaç başlangıcı). 4. `i <= N` şartını kontrol et. Şart yanlışsa Adım 7'ye git. 5. Ekrana `i * i` değerini yazdır. 6. `i = i + 1` yap ve Adım 4'e dön. 7. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[/N sayısını gir/]
    B --> C{{"i = 1; i <= N; i++"}}
    C -- Şart doğru --> D@{ shape: doc, label: "i * i değerini yazdır" }
    D --> C
    C -- Şart yanlış --> E([Bitir])
```

SORU 3: Bir Sayının Basamak Sayısını Bulma (For)

Kullanıcının girdiği pozitif tam sayının kaç basamaklı olduğunu bulun. Sayı her turda 10'a tam bölünerek küçültülür. (1254 için: 4)

Algoritma Adımları: 1. Başla. 2. Kullanıcıdan n sayısını al. 3. `sayac = 0` başlangıcıyla `n > 0` olduğu sürece döngüyü çalıştır (her turdan sonra sayac 1 artar). 4. Döngü gövdesinde `n = n / 10` yap (tam bölme) ve Adım 3'e dön. 5. Ekrana sayac değerini yazdır. 6. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[/Pozitif sayıyı gir: n/]
    B --> C{{"sayac = 0; n > 0; sayac++"}}
    C -- Şart doğru --> D["n = n / 10 (tam bölme)"]
    D --> C
    C -- Şart yanlış --> E@{ shape: doc, label: "sayac değerini yazdır" }
    E --> F([Bitir])
```

SORU 4: 0 Girilene Kadar Sayıların Ortalamasını Bulma (While + Karar)

Kullanıcı 0 girene kadar sayı almaya devam edin. 0 girildiğinde (0 hariç) girilen sayıların ortalamasını yazdırın. Hiç sayı girilmediyse ortalama hesaplanmaz.

Algoritma Adımları: 1. Başla. 2. `toplam = 0` ve `adet = 0` yap. 3. Kullanıcıdan `sayi` değerini al. 4. `sayi != 0` şartını kontrol et. Şart yanlışsa Adım 8'e git. 5. `toplam = toplam + sayi` yap. 6. `adet = adet + 1` yap. 7. Kullanıcıdan yeni `sayi` değerini al ve Adım 4'e dön. 8. `adet > 0` mı kontrol et. Değilse Adım 10'a git. 9. Ekrana `toplam / adet` değerini yazdır ve Adım 11'e git. 10. Ekrana "Sayı girilmedi" yazdır. 11. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[toplam = 0, adet = 0]
    B --> C[/Sayıyı gir: sayi/]
    C --> D{{"sayi != 0"}}
    D -- Şart doğru --> E[toplam = toplam + sayi]
    E --> F[adet = adet + 1]
    F --> G[/Sayıyı gir: sayi/]
    G --> D
    D -- Şart yanlış --> H{adet > 0 mı?}
    H -- Evet --> I@{ shape: doc, label: "toplam / adet değerini yazdır" }
    H -- Hayır --> J@{ shape: doc, label: "Sayı girilmedi" }
    I --> K([Bitir])
    J --> K
```

SORU 5: İlk 8 Fibonacci Sayısını Yazdırma (For)

Fibonacci dizisinde her sayı, kendinden önceki iki sayının toplamıdır. İlk 8 terimi yazdırın. (0, 1, 1, 2, 3, 5, 8, 13)

Algoritma Adımları: 1. Başla. 2. `a = 0` ve `b = 1` yap. 3. `i = 1` yap (sayaç başlangıcı). 4. `i <= 8` şartını kontrol et. Şart yanlışsa Adım 9'a git. 5. Ekrana a değerini yazdır. 6. `c = a + b` yap. 7. `a = b` ve `b = c` yap. 8. `i = i + 1` yap ve Adım 4'e dön. 9. Bitir.

Akış Diyagramı:

```
graph TD
    A([Başla]) --> B[a = 0, b = 1]
    B --> C{{"i = 1; i <= 8; i++"}}
    C -- Şart doğru --> D@{ shape: doc, label: "a değerini yazdır" }
    D --> E[c = a + b]
    E --> F[a = b]
    F --> G[b = c]
    G --> C
    C -- Şart yanlış --> H([Bitir])
```

---

4. Haftalık Özet

Beşinci haftamızı tamamlarken, artık herhangi bir kod komutu görmemiş olsanız bile bilgisayarların hangi mantıkla milyarlarca satır döngüyü sıfır hata ile çözdüğünün matematiksel "Düşünce" yapısına hakimsiniz. Bu diyagramları tasarlamak işin C, Python, C# ile kod yazma kısmından her daim daha önemlidir! Mimariyi iyi çizersek, ustalıkla tuğlaları üzerine ekleriz.

Alıştırma Sorusu: 1. Kendinize ait bir "Faktöriyel Hesaplayan" (Girilen 5 sayısı için 5x4x3x2x1 yapan algoritma) Akış Diyagramını defterinize paralel, kare, eşkenar çokgen ve okları kullanarak kendi limitlerinizle çizmeye çalışın... 2. İkinci soruyu düşünün: Herhangi bir Akış diyagramında Eşkenar dörtgenden "Çıkan Oklardan İkisi Birden (Hem Evet Hem Hayır)" tek bir noktada tekrar birleştirilebilir mi neden?

Hafta 5 Ödevi - Algoritma Tasarımı ve Temel Kodlama

Teslim Kuralları ve Önemli Uyarılar: * Bu ödevin cevaplarını içeren dosyanızı (PDF veya Word formatında), belirtilen son teslim tarihine kadar **Öğrenme Yönetim Sistemine (ÖYS/LMS)** yüklemeniz gerekmektedir. Zamanında sisteme yüklenmeyen ödevler sistem tarafından otomatik olarak kapatılacak ve değerlendirmeye alınmayacaktır. * Yapay zeka kullanımını doğrudan engellemek ve analitik düşünceyi ölçmek amacıyla; aşağıdaki **tüm sorular kişiye (Öğrenci Numaranıza) özeldir**. Başkasının verileriyle veya standart anonim verilerle yapılan yapay zeka kopyası ödevler sıfır (0) olarak değerlendirilecektir.

Soru 1: Sözde Kod (Pseudocode) Tasarımı (35 Puan)

Öğrenci numaralarının her bir hanesi sizin veritabanınızdır. Göreviniz şu: Kendi **kendi öğrenci numaranızın** rakamlarını tek tek inceleyen (örneğin 2100456 ise sırayla 2, 1, 0, 0, 4, 5, 6), bu rakamların içerisinden **sadece TEK sayı olanları (1, 3, 5, 7, 9)** tespit edip toplayan, çift sayıları ise es geçen ve kodun sonunda bu toplam değeri ekrana basan algoritmanın **SÖZDE KODUNU (Adım 1. Başla, Adım 2.. şeklinde)** yazınız.

Kontrol Adımı (Önemli): Sözde kodu yazdıktan sonra ödev kağıdınızın altına Öğrenci Numaranızı yazıp, tasarladığınız algoritmanın sizin numaranız girdiğinde **tam olarak hangi sonucu** bulacağını adım adım manuel (elle) hesaplayarak algoritmanın doğruluğunu teyit ediniz.

Soru 2: Akış Diyagramı (Flowchart) Tasarımı (35 Puan)

Bu soruda **"Kendi Öğrenci Numaranızın EN SON HANESİ"** sizin algoritma **"Zıplama/Artış Miktarınız"** olacaktır. (Uyarı: Eğer numaranızın son hanesi "0" (Sıfır) ise, sonsuz döngüye girmemesi için artış miktarını mecburen 5 (beş) olarak kabul ederek çözünüz).

Sıfırdan başlayarak, döngü her döndüğünde sayının üzerine kendi **artış miktarınız kadar** değer eklendiği ve bu toplam sayının **60'ı geçtiği anda durduğu** bir problemin **Akış Diyagramını** tasarlayınız. Örneğin, öğrenci numaranızın sonu 4 ise; 0, 4, 8, 12.. şeklinde sayılar üretilip toplanmalı, koşul 60'ı aşınca diyagram sonlandırılmalıdır.

- **Çözüm Yönetimi:** Akış diyagramınızı elinizle bir kağıda standart algoritma sembollerini (Paralelkenar, Eşkenar dörtgen, Dikdörtgen vb.) kullanarak çiziniz ve bu çizimin net okunabilir bir fotoğrafını ödev dosyanızın (Word/PDF) içine yerleştiriniz. Bilgisayar çizim araçlarını da (Draw.io, Lucidchart, Mermaid vb.) kullanabilirsiniz.

Soru 3: Kişiye Özel Döngü ve Sayı Sınıflandırma (30 Puan)

Bu soruda kullanacağınız değerleri kendi öğrenci numaranızdan türetiniz:

- **Artış miktarı (A):** Numaranızın son hanesi. Son hane `0` ise `A = 5` alın.
- **Bölen (B):** Numaranızın sondan ikinci hanesi. Bu hane `0` ise `B = 2` alın.

Algoritmanız `x = 0` değerinden başlamalı ve her turda önce x değerine `A` eklemelidir. Yeni x değeri **60'ı aşarsa**, bu değeri sınıflandırmadan döngüyü bitirmelidir. x değeri 60'ı aşmıyorsa aşağıdaki kurallara göre yalnızca bir gruba ekleyiniz:

1. x tek ise **Teklerin Toplamı** değişkenine ekleyiniz.
2. x çift ve `B` değerine tam bölünüyorsa **B'ye Bölünen Çiftlerin Toplamı** değişkenine ekleyiniz.
3. x çift ve `B` değerine tam bölünmüyorsa **B'ye Bölünmeyen Çiftlerin Toplamı** değişkenine ekleyiniz.

Döngü bittiğinde bu üç toplamı ekrana yazdırınız. Çözümünüzde:

1. Kendi numaranızdan `A` ve `B` değerlerini nasıl elde ettiğinizi gösteriniz.
2. Algoritmanın sözde kodunu adım adım yazınız.

3. Aynı algoritmanın akış diyagramını standart sembollerle çiziniz. Sözde kod ile diyagramın kararları ve işlem sırası birbiriyle aynı olmalıdır.
4. Her turdaki x değerini, hangi gruba girdiğini ve üç toplamın güncel değerlerini gösteren bir iz sürme tablosu hazırlayınız. 60'ı aşan ilk x değerini de tabloda gösteriniz; bu değer hiçbir toplama eklenmemelidir.

Puanlama: Sözde kod 10, akış diyagramı 12, kişisel değerlere göre doğru iz sürme ve sonuçlar 8 puandır.

Bireysel kontrol: Değerlendirme sırasında öğretmen, iz sürme tablonuzdan seçeceği bir turda x değerinin hangi gruba girdiğini ve toplamların nasıl güncellendiğini açıklamanızı isteyebilir.

Başarılar dileriz. Doğrudan hazır çözüm aramak yerine algoritmik düşünceyi kullanarak zihninizi yormanız, mesleki hayatınız için en değerli kazancınız olacaktır.

Hafta 6: C Programlama Dili: Değişkenler ve Veri Tipleri

1. Giriş ve C Diline Merhaba

Geçtiğimiz 5 hafta boyunca, programlama disiplininin arka planını hazırladık. Makinaların donanım/yazılım bağıntılarını, algoritmik düşünceyi ve süreç şablonlarını (Akış Diyagramlarını) irdeleyerek zihnimizi bir "yazılımcı analitiği" kıvamına soktuk. Bugünden itibaren bu dökümandan somut bir Programlama Dili olan **"C" Dili** dünyasına geçiş yapıyoruz. Artık yazdıklarımızı, ekrandaki programlara ve kod satırlarına dönüştüreceğiz.

Hedefler: - C dilinin çıkış amacı ve donanıma olan yakınlığını kavramak. - Temel bir C kod dosyasının mimarisini, ana şablonunu anlamak. - Ram'de yer alan veri tiplerinin (Tam Sayılar, Ondalık Sayılar, Karakterler) C dilindeki karşılıklarını detaylandırmak.

---

2. Neden C Programlama Dili Öğreniyoruz?

Önlisans ya da mühendislik seviyesindeki bir bilgisayar bilimi öğrencisinin temel yapı taşlarının başında C programlama dili öğretilir. Python gibi modern ve kısa komutlu diller çok popüler olsa da, programlama etiğini ve "arka planda bilgisayar bunu süzgecinden nasıl geçiriyor, belleği (RAM'i) nasıl harcıyor" sorusunu öğreten tek dil C dilleridir.

Pek çok modern dilin (Java, C#, C++) dedesi C'dir. C'de söz dizimini anlayan biri birkaç ay içerisinde dünyadaki çoğu teknik dili rahatlıkla idrak edecektir. Linux, Windows çekirdekleri, oyun motorları donanımın tam kalbine dokunabilme ve maksimum hızla çalışabilme kabiliyeti yüzünden çoğunlukla C & C++ tercihinde bulunmuştur.

---

3. C Derleyicisi (Compiler) ve İlk Programımızın İskeleti

3.1. C Derleyicisi (Compiler) Nedir?

Bizler programlarımızı insanların okuyup anlayabileceği İngilizce benzeri C komutlarıyla (örneğin `printf`, `int main`) yazarız. Ancak hatırlayacağınız üzere bilgisayar donanımı (işlemci) sadece `0` ve `1` adı verilen makine dilinden anlar. İşte yazdığımız bu C kaynak kodlarını (örneğin sonu `.c` uzantılı dosyaları) bilgisayarın anlayacağı makine diline çeviren, kodda kural hatası (syntax error) yoksa bunu çalıştırılabilir bir uygulamaya (Windows için `.exe` dosyasına) dönüştüren aracı yazılımlara **Derleyici (Compiler)** denir. GCC, Dev-C++ derleyici motoru veya Visual Studio bunlara örnektir. İşlem sırası basittir: Kod yazılır, derleyiciye verilir (compile işlemi) ve oluşan program çalıştırılır (run).

3.2. Kod İskeletimiz

C dilinde geliştirilmiş en temel, her "yeni başlayan"ın (Newbie) yazdığı ilk program ekrana merhaba yazdırmakla başlar. İncelemek üzere o kodu sunalım:

```
#include <stdio.h> // Birinci kisim: Kutuphaneler dahil ediliyor.

// Ikinci kisim: Ana (Merkez) Fonksiyon. Program calismaya buradan baslar.
int main() {

    // Ucuncu kisim: Kodlarin - Gozle Gorulen Komutlarin Basladigi Alan!
    printf("Merhaba, Programa Hazirim!\n");

    return 0; // Komut satiri duzgun calisti bilgisini sisteme ilet ve fonksiyonu kapat.
}
```

Bu iskelet parçalarındaki temel bilgiler şöyledir: - `#include`: Standart "Ekrandan Veri Almak ve Ekrana Veri Yazdırmak (Input/Output)" için `stdio.h` isimli kütüphaneyi çekmemizi sağlar. Bu çekilmezse, ekrana kelime yazma komutu olan `printf` kendi görevini tanımazdı! - `int main()`: Tüm C dillerindeki uygulamalarda, bilgisayar derlemeye / kurgulamaya direkt olarak main kelimesini arayarak başlar. Bu bloğun içine de işlem akışınızı eklersiniz. - { ve } (Süslü Parantezler): Türkçe düşünürsek "Birleşik bir cümlelik (Kapsam)" demektir, yazılan satırlar parantezin sınırını terk etmez! - Noktalı virgül `;`: Her komutun yan yana bittiğini anlatan, Türkçe gramerlerdeki "Nokta(.)" yapısıdır. Bunlar unutulursa sistem kodu derlemeden kırmızı hata basar (SyntaX - Gramer Hatası).

---

4. Değişkenler ve Bellek (RAM) Yönetimi

Akış diyagramları kısmında `Sayı 1`, `Sayı 2` şeklinde kafamızdan metinle isimlendirerek kullandığımız değişkenler kısmına döndük. Fakat C dili çok katı bir dildir. Sisteme "Selam ben `Sayi` diye değişken tanımladım" diyip geçemezseniz! O verinin **"NE TİPTE/BOYUTTA"** olacağını da (bu bir kelime mi, harf mi, bin küsürlü atamalı bir sayı mı vs.) ona önceden haber vermelisiniz. Haber vermelisiniz ki bilgisayar, geçici RAM belleğinde ihtiyaçtan büyük yerleri rezerve edip bilgisayarı çökertmesin.

İşte bu sürece C dilinde **Değişken Belirleme ve Veri Tipleri (Data Types)** deniyor.

C Dilinde Değişken Tanımlama Kuralları: 1. **Önce tanımla, sonra kullan:** Bir değişken kullanılmadan önce veri tipiyle birlikte tanımlanmalıdır. (`int yas;` yazmadan `yas = 20;` yazılamaz.) 2. **Veri tipi zorunludur:** Her değişkenin tipi (`int`, `float`, `char`...) başta belirtilir ve sonradan değiştirilemez. 3. **Değişken adı harfle veya alt çizgiyle başlamalıdır:** `sayi1`, `_toplam` geçerlidir; `1sayi` geçersizdir. Değişken adı sayı ile başlamamalıdır. 4. **Değişken adı yalnızca harf, rakam ve alt çizgi (_) kullanılabilir:** Boşluk, tire (`-`) ve özel karakterler (`@`, `#`, `$`, `%` vb.) kullanılamaz. (`not ortalamasi` ve `ders-notu` geçersizdir; `not_ortalamasi` geçerlidir.) 5. **Türkçe karakter kullanılmamalıdır:** `ç, ğ, ı, ö, ş, ü` yerine `c, g, i, o, s, u` kullanın. (`ogrenciYasi` yazın, `öğrenciYaşı` yazmayın.) 6. **Büyük/küçük harfe duyarlıdır (Case Sensitive):** `yas`, `Yas` ve `YAS` üç ayrı değişkendir. 7. **Anahtar kelimeler değişken adı olarak kullanılamaz:** `int`, `float`, `char`, `return`, `if`, `while`, `for` gibi C'nin ayrılmış kelimeleri değişken adı olamaz. 8. **Aynı isim aynı kapsamda iki kez tanımlanamaz:** Bir `main` içinde iki kez `int sayi;` yazılırsa hata verir. 9. **Her değişken tanımlama noktalı virgülle biter (;).** 10. **Başlangıç değeri vermek iyi alışkanlıktır:** Değer verilmeyen değişkenin içinde rastgele (çöp) bir değer bulunabilir. (`int toplam = 0;`) 11. **Anlamlı isim seçin:** `a`, `x` yerine `yas`, `toplam`, `ortalama` gibi ne tuttuğu anlaşılan isimler kullanın.

Doğru / Yanlış Örnekler:

```
int yas;              // Doğru
float not_ortalamasi; // Doğru
char _harf = 'A';     // Doğru
int toplam = 0;       // Doğru (başlangıç değeriyle)

int 2sayi;            // YANLIŞ: Rakamla başlıyor
float not ortalamasi; // YANLIŞ: Boşluk var
int ders-notu;        // YANLIŞ: Tire var
int return;           // YANLIŞ: Anahtar kelime
```

4.1. Temel Veri Tipleri

1. **`int` (Integer - Tamsayılar):** Kesir / Bölünmeyen kaba sayılardır. (-5, 10, 0, 1000). Ram'de (genellikle) 4 Byte yer kaplarlar -2 Milyar ila +2 Milyar arasındaki sayılara kucak açabilir. Tanımlama örneği: `int Sayi = 5;`, `int Yas = 21;`

2. **`float` (Ondalık/Kesirli Sayılar):** Küçük virgüllü ve noktalı rakamlarda kullanılan sayılardır. Tanımlama örneği: `float Maliyet = 15.5;` (Not: Türkiye'deki gibi virgül (,) KULLANILMAZ sistemde nokta (.) basılır).

3. **`double` (Kapasiteli Ondalık Sayılar):** Float'ın büyük kardeşidir. Bilimsel hesaplar gibi küsürat hatasının istenmediği durumlarda, daha fazla bellek kullanan ama daha hassas olan bu tipi seçersiniz (yaklaşık 15 haneye kadar hassasiyet, 8 Byte). Tanımlama örneği: `double pDizisi = 3.14159265;`

4. **`char` (Character - Sadece Tek Bir Karakter):** Klavyedeki harf ('A', 'B') sembol veya sayı değerlerini ancak "Teker teker" saklamak için yaratılmış tek byte alan tiplerdir. En belirgin özelliği karakterler içeri atanırken değer Tırnak (`'...'`) içinde verilmelidir. Tanımlama Örneği: `char KanGrubu = 'A';`

5. **`string` (Metin / Karakter Dizisi):** Birden fazla karakterin yan yana gelmesiyle oluşan kelime ve cümlelerdir. **C dilinde `string` adında hazır bir veri tipi yoktur;** metin, `char` dizisi (`char[]`) ile tutulur ve değer çift tırnak (`"..."`) içinde verilir. (Python, C#, Java gibi dillerde `string` doğrudan kullanılır.) Diziler konusunu ileri haftalarda ayrıntılı işleyeceğiz, şimdilik kalıp olarak bilmeniz yeterlidir. Tanımlama örneği: `char Ad[] = "Ayşe";`, `char Sehir[20] = "Ankara";` (Tek tırnak `'A'` tek karakter (char), çift tırnak `"A"` metindir (string). İkisi birbirinin yerine kullanılamaz.)

6. **`bool` (Mantıksal - Doğru / Yanlış):** Yalnızca iki değer alabilen tiptir: `true` (doğru) ve `false` (yanlış). Karar yapılarındaki "Evet/Hayır" cevaplarının karşılığıdır. C dilinde `bool` kullanmak için `#include <stdbool.h>` kütüphanesini eklemek gerekir. Aslında arka planda `true` = 1, `false` = 0 olarak saklanır. Tanımlama örneği: `bool GirisYapti = true;`, `bool Gectimi = false;`

```
#include <stdio.h>
#include <stdbool.h> // bool tipi için gerekli

int main() {
    char Ad[] = "Ayşe";
    bool Gectimi = true;
    return 0;
}
```

4.2. Değişken Tanımlama Süreç Formülü

C dilinde bir değişkeni oluştururken soldan sağa bir formül vardır: `Veri_Tipi_Adı` + `Kullanıcının Verdiği Metinsel Değişken Adı` + `=` + `Değeri` + `;`

Ayrı Ayrı Kod Parçalarına Yansımaları:

```
int main() {
    // 1- İsterseniz kutuyu önceden açıp, değerini sonra yerleştirebilirsiniz.
    int ogrenciYasi;
    ogrenciYasi = 23;

    // 2- İsterseniz "Kutunu ismini verirken peşinen değer takabilmek de yasal!"
    float biskivuFiyati = 9.99;
    char okulKodHarfi = 'M';

    // Ayni turden birden cok cift kutucuk acilabilir..
    int Vize, Final, Ortalama;

    return 0;
}
```

Burada en hassas kural: C "Büyük ile Küçük" Harfe Duyarlıdır! (Case Sensitive). Eğer programda `int a` derseniz ve aşağıda A değerini kullanmaya çalışırsanız program o değişkeni bulamaz. C'ye göre (a) ve (A) aynı anlamlara gelmez!

---

5. Değişkenleri Ekrana Bastırmak Üzerine Küçük Bir Başlangıç (% Parametreleri)

Biz `printf("Merhaba\n")` diyerek tırnak içini normal basarız. Peki demin oluşturduğumuz yaşı (ogrenciYasi= 23) ekranda "Kişinin yaşı: 23" yazmak istersek ne yapacağız? `printf("Yas: ogrenciYasi")` dersek ekranda literal o metni yazar 23 basmaz.

İşte değişkenin içeriğini C'de ekranda göstermek istiyorsanız "Biçim Niteleyicileri (Format Specifiers)" gerekir (Bunlara format damgası denir): - `%d` veya `%i` : Tamsayıları (int) temsil eder. - `%f` : Ondalık yapıları (float) simgeler. - `%c` : Harfleri / karakterleri (char) çeker. - `%s` : Metinleri (karakter dizisi / string) yazdırır. Örnek: `printf("Ad: %s", Ad);` - `bool` için ayrı bir niteleyici yoktur; `true`/`false` 1/0 olarak saklandığı için `%d` kullanılır. Örnek: `printf("%d", Gectimi);` (ekrana 1 yazar)

```
#include <stdio.h>

int main() {
    int Yil = 2026;
    float Bakiye = 120.50;

    // Yili virgüllü yapı ile C'ye çektirdik..
    printf("Bulundugumuz yil: %d dir.\n", Yil);

    // Küsüratlı verimizi bir de çektirelim...
    printf("Elimizdeki Paramiz (Bakiye) : %f Turk Lirasidir.\n", Bakiye);
    return 0;
}
```

---

6. Haftalık Özet

Bu hafta diyagramların hayal aleminden çıkıp, gerçek kod kütüphanesi olan `stdio.h` 'ı bağlamayı, `printf()` kelimesiyle komuta dökmeyi kavramaya başladık. Operatörler, algoritma derslerinde gördüğünüz ve formüle ettiğiniz Matematiksel işaretlerin birebirleri olduğundan o adımları aynı harflerle koda koyabilirsiniz (`int toplam = x + c;`gibi). Bizim asıl eforumuz; C dilindeki en hayati iskelet olan "RAM'in limitlerini gözetmek adına Veri Tipinin İllaki (Zorunlu Olarak) Belirtilmesi (`int`, `float`, `char`)" sürecidir. İleride sistemden veri girerken bile (Kullanıcı yaşını söyle demeden evvel) daima programın başında bu kutucukları önceden ayırmayı adımız kadar iyi bilmeliyiz!

Alıştırma Testleri: 1. Kişisel bilgisayarına, programlamaya uygun C kodu derleyen bir Ide program kur (Visual Studio Code, Dev-C++, veya CodeBlocks) ve ilk koddaki "Merhaba, Programa Hazirim!" metnini bastırarak konsol ortamını (Siyah Ekran) gözlemle. 2. Ondalıklı (`float` yapısını) kullanan bir veri ile `printf` metodu birleştirildiğinde, değer olarak virgülden sonra istemediğimiz kadar 0 rakamı ortaya çıkabilir! Format basamağını `%0.2f` yaptığınızda ekranda ne değişmektedir, araştırıp uygulayınız.

Hafta 6 Ödevi - Değişken İsimlendirme Kuralları

Soru 1: Değişken İsimlendirme Kuralları (30 Puan)

Aşağıdaki cümlelerde boş bırakılan yerleri uygun sözcüklerle tamamlayınız. Ardından verilen değişken adlarının her birini **Geçerli** veya **Geçersiz** olarak işaretleyiniz. Geçersiz olan her ad için hangi kurala uymadığını gerekçesiyle açıklayınız.

1. C dilinde bir değişken adı bir harf veya _____ ile başlayabilir; _____ ile başlayamaz.
2. Değişken adının devamında harf, _____ ve alt çizgi (_) kullanılabilir.
3. Değişken adlarında boşluk ve _____ gibi özel karakterler kullanılamaz.
4. C dilinde `int` gibi _____ sözcükler değişken adı olarak kullanılamaz.
5. C dili büyük ve küçük harfe _____ olduğundan yas ve Yas _____ değişken adlarıdır.

Değişken adı	Geçerli / Geçersiz	Geçersizse gerekçesi
`ogrenciYasi`	_____	_____
`2ogrenci`	_____	_____
`ogrenci-yasi`	_____	_____
`int`	_____	_____

Hafta 7: Akış Diyagramlarından Kodlamaya: Giriş-Çıkış Komutları ve Karar Yapıları

1. Giriş ve Ön Hazırlık

Geçen dersimizde, C dilinin çatısını tanımış ve programlama evrenindeki belleklere uygun verilerin kutuya nasıl (int, float, vb) atanacağını, ve bunları ekrana `printf` yardımıyla yüzde işaretleriyle nasıl yansıtacağımızı aktardık. Fakat sadece ekrana yazı yazdırmak statik, cansız ve tek düze bir yapı sunar. Programlar "Kullanıcı"lara ihtiyaç duyar. İnsanlarla iletişim kurmaları için "Sorular Sorabilmeli ve Veri Alabilmeleri (Input)" gerekir. Ayrıca geçen haftalarda defalarca "Akış Diyagramlarına" işlediğimiz "Eşkenar Karar" kutularını bugün gerçek kelimelerle, hakiki "Karar (If - Else)" şart cümlelerine oturtacağız. Akış Diyagramı serüveni kodlu hayata vücut bularak giriş yapacaktır!

Hedeflerimiz: - C programlama dilindeki veri alma-okuma (`scanf`) komutunun mantığını oturtmak. - `if`, `else if` ve `else` komutlarıyla şartlı dallanma yapabilmek. - Tasarlanan bir Karar Akış Diyagramı şemasını kendi kod satırlarına hatasız işlemek.

---

2. Kullanıcı Girişi Almak (Girdi - `scanf`)

Karşı tarafta klavyede olan kullanıcıdan değer almak ve bunu kutunun (değişkenin) içine koymak C programında **`scanf()`** anahtar kelimesiyle başarılır. `Scanf`, aynen `Printf` gibi kütüphanesi hazır (stdio.h) olarak tarafımızdan gelmektedir!

Farketmişsinizdir ki, çıktıda yaptığımız, int veya karakter için olan özel `%` formatlarını, bu defada "Hangi stilde bir değeri kutuya koyacağız" manasıyla klavyeye uyarlıyoruz.

Sözdizimi Yapısı: `scanf("Format Kodu Türü", & Değişkeninizin Adı);`

Not: C dilindeki veri okuma sisteminde "Adres/Yerbelirtme ampersand işareti" olan **&** kesinlikle kullanılmak zorundadır (Yoksa kullanıcı giriş yapsa dahi hata yaratır, bilgisayara veriyi fırlatmayı engeller). Anlamı şudur: "Klavyeden okunan değeri, bellekte isimlendirdiğimiz Değişkeninizin **ADRESİNE** götür yapıştır".

```
#include <stdio.h>
int main() {
    int OgrenciNo;

    // Oncelikle biz kullaciya aciklayici yazi ciktisi veriyoruz ki kullanici napacagini
bilsin:
    printf("Lutfen sistem icin Ogrenci Numaranizi (Tam sayi) giriniz:\n");

    // Klavyeden girise ve o girisi adres ( & ) yardimiyla kutuya hapsetmeye haziriz:
    scanf("%d", &OgrenciNo);

    printf("Harika! Sisteme Girdiginiz Ogradiginiz Numarasi ==> %d <-- dir..\n",
OgrenciNo);

    return 0;
}
```

---

3. Karar Yapılarının C Dili Karşılıkları (Eşkenar Çokgenin Kodlanması)

Program kodlarken, Akış Diagramlarında karar vermek için "Eğer durum böyleyse sağdaki kola (Evet) git, Değilse soldaki (Hayır) kola geç" diyoruz. C dünyasındaki o eşkenar çokgenin adı aynen algoritmada dilleştirdiğimiz haliyle **`if`** bloğu, hayır kolu ise **`else`** bloğudur.

3.1. Temel Bir `if` ve `else` Çatısı

Eğer sorgulanacak blok **DOGRU/TRUE/EVET** dönerse çalışmasını fırlatacağı yer `if`'in parantezleridir. Aynı eşkenar döngü sorgulanıp **HATA/YANLIS/HAYIR** olarak dönüş fırlatılırsa otomatik bir sonraki `else` in parantezlerine gider.

```
// Formul Yapisi
if ( SART VEYA KOSULLARI MATEMATIKSEL OPERATORLERLE YAZDIGIN YER) {
    // SART SAGLANIYORSA BURADAKI SATIRLARA GIR, KODU BURADAN ÇALIŞTIR !
} else {
    // SART SAGLANMIYORSA, C PROGRAMI DIREKT BURAYA SIÇRAYACAKTIR..
}
```

Küçük bir vize uygulamasını gerçek koda çeviriyoruz; Geçme Notu 60 olmak zorundaydı.

```
#include <stdio.h>

int main() {
    int  OrtalamaNot;

    printf("Dersten aldiginiz Notu Giriniz: ");
    scanf("%d", &OrtalamaNot);

    // Karsilastirma / İliski Operatorlerinin Hatirlanilamasi (> ,  <,  == ,  >=)
    if (OrtalamaNot >= 60) {
        // Eger giren notu gercekten karsilastirmayi sagliyorsa, tebrik eder:
        printf("Tebrikler. C dilinden Gectiniz!\n");
    } else {
         // Ya esit veya buyuk DEGILDI.. Cihaz bunu algiladigi an uste girmez..
         printf("Uzgunuz.. Siniftan Kaldiniz..\n");
    }

    return 0;
}
```

3.2. İç İçe Geçmiş Seçenekler ve `else if` Merdiveni

Diyagram derslerinde, iki sayıyı veya sistemi ayırırken "0 mı ? Yoksa Sıfırdan Büyük mü ? O Zaman ikisi de tutmadıysa Negatiftir (Sıfırdan Küçüktür)" deriz. Sadece EVET/HAYIR 2 tane kapıdan oluşmaz. Birden çok ifli soruyu birbirinin ardına ekranda bağlamaya yarayan sisteme **else if (Değilse Ama Yeni Bir Soru Sor/Eğer..) merdiveni** denilir.

(Not: Sayı 2'ye bölününce kalanı SIFIR ediyorsa Eşittir manasında "==" işleçleriyle yapılır)

```
int main() {
    int girilenX;

    printf("Karsilastirmalik Bir Deger: ");
    scanf("%d", &girilenX);

    if (girilenX > 0) {
        printf("Bilgi: Sayi POZİTIF karakterli!");
    }
    else if (girilenX < 0) {
        // 2. KAPI - Ilki Eger degilse, bir de suna sorsun!
        printf("Bilgi: Sayi NEGATIF karakterli!");
    }
    else {
        // Ustteki iki kapi da kirildi/yanlis. Ihtimal artik YOK (Geride sadece 0
```

```
olabilir.. Sonuc olarak sadece ELSE kalmistir.)
        printf("Bilgi: Sistem SIFIR(0)'dir..");
    }

    return 0;
}
```

Ek Detay: Algoritmalarda Mantıksal operatörler işledik; VE (*&&*) VEYA (||). İşte *`if( Not>=0 && Not<=100)`* yazarak (C programında VE *&&* ile VEYA || ile belirtilir) her iki kuralı da aynı *`if`* satırına hapsedebileceğimizi unutmayınız.

---

4. Akış Diyagramını Koda Dönüştürme Taslağının "Canlandırılması"

Derste ilk öğrendiğiniz "Maaş Artışı Kararı", bu örneğini bir Algoritmasından Kurguya ve C Diline adım adım bağlaması: **A.** "Kullanıcı dışardan bir Maaş tutarı girecek, Eğer tutar eski ve düşük olan 17000'dan azsa veya ona "==" Eşitse Zammı (%20) olarak maaşına eklesin Ekrana Bastırsın. Aksi taktirde büyükse %10 eklensin ve bassın"

```
#include <stdio.h>
int main() {
   float alinanMaas;
   printf("Elemaninizin Massini Tl cinsinden kaba girin:\n");
   scanf("%f", &alinanMaas);

   if(alinanMaas <= 17000.0) {
        //  0.20 carpmak = %20 demektir.. Eski massı guncelle..
        alinanMaas = alinanMaas + (alinanMaas * 0.20);
        printf("Cikan Yeni Maas= %f", alinanMaas);
   } else {
        // Yoksa  >17000 durumunda  zaten ust asilacagindan:
        alinanMaas = alinanMaas + (alinanMaas * 0.10);
        printf("Artis Kismenn de olsa uyarlandi : %f", alinanMaas);
   }
   return 0;
}
```

---

5. Haftalık Özet

Beş ve altıncı haftada, ekrana çıktı vermek ("Merhaba C") ile kapattığımız sürecimiz; bu yedinci eğitim dilimi ile interaktif ve konuşkan (Soru cevap – `Scanf`) duruma çıkmıştır. Karar, eşkenar ve Algoritmik Süzgeç yapısını oluşturan "EVET-HAYIR" döngüsü; `if - else` sayesinde gerçeğe, derleyici mekanizmasına intikal etmiş olup bir sonraki adım "döngü algoritma yansımalarıdır".

Uygulama/Alıştırma Egzersizleri: 1. C editörünü açarak, sistem ekranına `Vize 1`, `Vize 2`, ve `Final` şeklinde 3 ayrı int girişi oluşturan öğrenci sistemi yazın (Üçünde de `scanf` istemesi yapılsın). Formüle bu 3 sayıyı hesaplayarak yansıtsın ve 50 den büyükse if kararından tebriği okutsun.. 2. Klavyeden aldığı "1" girişine "Pazartesi", "2" girişine "Salı"... yapacak C karar programını (`if`, `else if`, `else if` ...şeklinde ardı ardına sıralayıp) sonuna kadar oluşturunuz ("else" için geçersiz numara diyebilirsiniz).